

Spring Framework – Notes

~Yash Pratap Singh

1. Section Overview

Goal: Build a **Loosely Coupled Hello World Gaming App** using Spring, and learn core terminology: *Tight Coupling, Loose Coupling, IOC Container, Application Context, Component Scan, Dependency Injection, Spring Beans, Auto Wiring*

Approach — 3 Iterations (GameRunner class running Mario, SuperContra, PacMan):

- **Iteration 1:** Basic GameRunner + Game classes -> **Tightly Coupled**
- **Iteration 2:** Introduce **Interfaces** (GamingConsole) -> **Loose Coupling Level-1**
- **Iteration 3:** Spring Framework manages objects -> **Loose Coupling Level-2**

2. Creating a Spring Project (Spring Initializr)

Site: start.spring.io

Setting	Value
Build Tool	Maven
Language	Java
Spring Boot	Latest stable (avoid SNAPSHOT versions)
Group	com.in28minutes.spring
Artifact	learn-spring-framework
Java version	17+ (required for Spring Boot 3)

Steps:

1. Choose options -> click **GENERATE** -> zip downloads
2. Unzip to local folder

3. Eclipse -> File -> Import -> Existing Maven Projects -> select folder -> **Finish**
4. Verify pom.xml shows correct Group/Artifact
5. Expand project -> check src/main/java, src/main/resources, src/test/java, Maven Dependencies

Tip: Always use latest Eclipse (Java EE edition) to avoid setup issues.

3. Iteration 1 — Tightly Coupled Code

Creating MarioGame + GameRunner

```
// MarioGame.java
public class MarioGame {
    public void up() {
        System.out.println("up");
    }
    public void down() {
        System.out.println("down");
    }
    public void left() {
        System.out.println("left");
    }
    public void right() {
        System.out.println("right");
    }
}

// GameRunner.java
public class GameRunner {
    private MarioGame game;
    public GameRunner(MarioGame game) {
        this.game = game;
    }
    public void run() {
        game.up(); game.down(); game.left();
        game.right();
    }
}
```

Both classes go in a **sub-package** (...learnspringframework.game) of the main application package.

Adding SuperContraGame

- Copy-paste MarioGame -> rename to SuperContraGame

- To switch GameRunner from Mario to SuperContra: **must manually edit GameRunner.java** (change type) and the main class (change object creation)

Key Problem — Tight Coupling

Every time you switch games, you must **modify GameRunner's source code**. GameRunner is *hardcoded* to depend on one specific game class.

Definition: *Tight Coupling* = a class directly depends on a concrete implementation, so any change forces code changes in the dependent class too.

Fix (next step): Use **Interfaces** to decouple GameRunner from specific game classes.

4. Iteration 2 — Introducing Interfaces (Loose Coupling Level-1)

Problem to solve

GameRunner was tightly coupled to one specific game class. Goal: remove that dependency using a common **interface**.

Creating the GamingConsole interface

All games (Mario, SuperContra, PacMan) perform the same actions: up, down, left, right. Extract these into an interface.

```
java
// GamingConsole.java
public interface GamingConsole {
    void up();
    void down();
    void left();
    void right();
}
```

Making games implement the interface

```
java
public class MarioGame implements
GamingConsole {
    public void up() {
        System.out.println("up"); }
}
```

```
    public void down() {
        System.out.println("down"); }

    public void left() {
        System.out.println("left"); }

    public void right() {
        System.out.println("right"); }
}
```

```
public class SuperContraGame implements
GamingConsole { ... }
```

```
public class PacmanGame implements
GamingConsole { ... }
```

Updating GameRunner to depend on the interface, not a concrete class

```
java
public class GameRunner {
    private GamingConsole game;

    public GameRunner(GamingConsole game) {
        this.game = game; }

    public void run() {
        game.up(); game.down(); game.left();
        game.right();
    }
}
```

Result

In the main class, you can now switch games **without touching GameRunner.java** at all:

```
java
GamingConsole game = new MarioGame();
// or SuperContraGame(), or PacmanGame()

GameRunner runner = new GameRunner(game);
runner.run();
```

Only the object creation line changes (and the import). GameRunner itself stays untouched — it just depends on the GamingConsole type.

Key Concept — Loose Coupling (Level 1)

GameRunner now depends on an **abstraction** (GamingConsole interface), not a **concrete implementation** (MarioGame, SuperContraGame, etc.).

Definition: *Loose Coupling* = a class depends on an interface/abstraction rather than a

specific implementation, so implementations can be swapped without modifying the dependent class.

Exercise done

Created PacmanGame implements GamingConsole and ran it via the same unchanged GameRunner — confirms the interface-based decoupling works.

What's still manual

You are still writing new MarioGame() / new GameRunner(game) yourself in code — object creation and wiring is manual.

Next (Iteration 3): Spring Framework will take over **creating and wiring these objects** for you → **Loose Coupling Level-2.**

5. Iteration 3 — Spring Framework Manages Objects (Loose Coupling Level-2)

Problem before Spring

To create GameRunner, you manually:

1. Create implementation of GamingConsole (e.g., MarioGame)
2. Pass it as constructor argument to GameRunner

This is manual **object creation + dependency management**. Fine for 2 classes — but real enterprise apps have **thousands of classes with dependencies**. Spring Framework can take over this job.

Step 1: Mark classes as Spring-managed components

```
java
// MarioGame.java

import
org.springframework.stereotype.Component;
```

```
@Component

public class MarioGame implements
GamingConsole {

    public void up() {
        System.out.println("jump"); }
}
```

```
    public void down() {
        System.out.println("down into a hole"); }

    public void left() {
        System.out.println("stop"); }

    public void right() {
        System.out.println("accelerate"); }
}

java
// GameRunner.java

import
org.springframework.stereotype.Component;

import
org.springframework.beans.factory.annotation
.Autowired;

@Component

public class GameRunner {

    private GamingConsole game;

    @Autowired

    public GameRunner(GamingConsole game) {

        this.game = game;
    }

    public void run() {

        game.up(); game.down(); game.left();
        game.right();
    }
}
```

What each annotation means:

- @Component -> tells Spring "manage an instance (bean) of this class"
- @Autowired -> tells Spring "automatically inject/wire the required dependency here"

Step 2: Launch the Spring Application Context

```
java
// LearnSpringFrameworkApplication.java

@SpringBootApplication

public class LearnSpringFrameworkApplication
{
}
```

```

    public static void main(String[] args) {
        ConfigurableApplicationContext
context =

SpringApplication.run(LearnSpringFrameworkAp
plication.class, args);

        GameRunner runner =
context.getBean(GameRunner.class);

        runner.run();

    }
}

```

Output: jump, down into a hole, stop, accelerate

Key Terminology (now demonstrated in code)

Term	Meaning
Component	A class Spring manages, marked with @Component
Bean	An actual instance/object created and managed by Spring (instance of a component)
Application Context (Spring Context)	The container that holds and manages all beans and their dependencies
Dependency	An object a class needs to function (e.g., GamingConsole is a dependency of GameRunner)
Dependency Injection (DI)	Spring automatically supplying (injecting) a required dependency into a class
Autowiring	The mechanism/annotation (@Autowired) Spring uses to perform dependency injection
IOC Container (Inversion of Control)	Spring itself — control of object creation is "inverted" from the developer to the framework

How context.getBean() works

java

```
context.getBean(GameRunner.class)
```

Asks the Spring context: "give me the managed instance (bean) of type GameRunner." Spring already created it — along with its dependency MarioGame — when the context launched.

Open Questions (raised, to be answered in later steps)

1. What's happening in the background when Spring creates these objects?
2. How does this relate to the terminology discussed earlier — Dependencies, Dependency Injection, IOC Container, Application Context, Component Scan, Spring Beans, Auto Wiring?
3. Does Spring Framework really add value — we replaced 3 simple lines with 3 "more complex" lines?
4. What if I want to run SuperContraGame instead of MarioGame — what change is needed?
5. How is the Spring Jar downloaded (which Maven dependency brings Spring in)?

6. Answer to Q1 — What's Happening in the Background?

Enabling DEBUG logs

To see what Spring is doing internally, increase log level in application.properties:

properties

```
logging.level.org.springframework=debug
```

Note: exact debug log format changes across Spring versions — don't worry if yours differs slightly.

What the DEBUG log reveals

Identified candidate component class: file [GameRunner.class]

Identified candidate component class: file [MarioGame.class]

Creating shared instance of singleton bean
'gameRunner'

Creating shared instance of singleton bean
'marioGame'

Autowiring by type from bean name
'gameRunner' via constructor to bean named
'marioGame'

Interpretation:

- Spring **scans** for classes with
@Component -> calls them "candidate
components"
- It **creates instances** (beans) of each ->
by default, each bean is a **Singleton**
(only one instance per Spring context,
unless configured otherwise)
- It detects GameRunner's constructor
needs a GamingConsole -> finds
MarioGame implements it -> **autowires
via constructor**

Background flow: Spring scans for
@Component classes -> identifies
dependencies of each -> creates instances ->
injects (autowires) dependencies into
constructors -> manages all these beans inside
the Application Context.

Experiment 1: Remove @Component from MarioGame

No class implements GamingConsole as a
component anymore.

Error:

Parameter 0 of constructor in GameRunner
required a bean of type

'GamingConsole' that could not be found.

Action: Consider defining a bean of type
'GamingConsole' in your configuration.

Meaning: Spring needs a GamingConsole
implementation to inject into GameRunner, but
none is marked @Component — so it has **zero
candidates** and fails.

Fix: Add @Component to *any one*
implementation (e.g., PacmanGame) → works
again.

Experiment 2: Add @Component to TWO implementations (e.g., PacmanGame + SuperContraGame)

Error:

GameRunner required a single bean, but 2
were found:

pacmanGame, superContraGame

Action: Consider marking one bean as
@Primary

Meaning: Spring found **multiple matching
beans** for one dependency slot and doesn't
know which to inject — this is ambiguous, so it
throws an error.

Fix (suggested by Spring): Mark one
implementation @Primary, or remove
@Component from the extras, so only **one**
candidate remains.

Key Rule Learned

For @Autowired to work without error, Spring
needs **exactly one matching bean** for each
dependency:

Situation	Result
0 matching @Component beans	Error — "no bean of type X found"
1 matching @Component bean	Works — autowired correctly
2+ matching @Component beans	Error — "expected single bean, found N" (fix: use @Primary)

Final working state (for this step)

- @Component only on MarioGame
(among game classes)
- @Component on GameRunner

- Everything else has @Component removed
- App runs MarioGame successfully via Spring-managed beans

7. Answer to Q2 — Terminology Explained

Component

A class managed by Spring Framework.

Marked with @Component (other similar annotations: @Service, @Repository — but @Component is the base/most important one).

Dependency

Something a class **needs to be created**.

Example: GameRunner needs a GamingConsole implementation (e.g., MarioGame) -> so MarioGame is a **dependency** of GameRunner.

Component Scan

Defines **which packages Spring searches** for @Component classes. Spring cannot scan the entire codebase — only configured packages.

- @SpringBootApplication automatically includes @ComponentScan on the **current package** (and its sub-packages) by default.
- Example: main class is in com.in28minutes.spring.learnspringframework, so everything in that package and sub-packages (like ...learnspringframework.game) is scanned automatically.

Custom / multiple package scanning:

java

```
@ComponentScan({"com.package1",
"com.package2"})
```

Common error — "No bean of type X found":

Usually caused by one of two mistakes:

1. @ComponentScan misconfigured — component class lies **outside** the scanned package
2. The class is missing the @Component annotation entirely

(Demonstrated by pointing @ComponentScan at a non-existent package -> GameRunner bean not found -> error.)

Dependency Injection (DI)

The process of:

1. Identifying beans
2. Identifying their dependencies
3. **Wiring/injecting** those dependencies automatically

This is exactly what Spring does — which is why Spring is called a **DI Framework**.

Inversion of Control (IoC)

Before Spring: **you** wrote code to create objects (new MarioGame(), etc.) — you were in control. With Spring: **the framework** creates and manages objects for you.

Definition: Control over object creation/wiring is "inverted" — shifted from your code to the framework. This is why DI is also called **Inversion of Control**.

Spring Beans

Objects whose **instance and lifecycle are managed by Spring** (created because of @Component). Example: the GameRunner and MarioGame instances Spring creates are beans.

IoC Container

The technical component **inside Spring that manages beans and their dependencies/lifecycle**.

Two types:

Type	Usage
ApplicationContext	Most commonly used (default choice in real-world apps)
BeanFactory	Rarely used in practice

Launching a SpringApplicationContext = launching an IoC container (specifically, an ApplicationContext).

Autowiring

The process of **identifying a bean's dependencies and wiring them together automatically**.

Quick Reference Table

Term	One-line Definition
Component	A class managed by Spring (@Component)
Dependency	Something a class needs to be instantiated
Component Scan	Defines which packages Spring searches for components
Dependency Injection	Identifying beans + dependencies, then wiring them
Inversion of Control	Object-creation control shifts from developer to framework
Spring Bean	An instance of a component, managed by Spring
IoC Container	Spring's internal engine managing beans (usually ApplicationContext)
Autowiring	Auto-identifying and injecting a bean's dependencies

8. Answer to Q3 — Does Spring Framework Really Add Value?

The real-world problem

Small apps (like our Game example) don't show Spring's real power. **Enterprise applications** have:

- **Multiple layers:** Web Layer -> Business Layer -> Data Layer
- **Thousands of classes**, each with dependencies on classes in the layer below
- Example: BusinessService depends on DataService; WebController depends on BusinessService

Without Spring, you'd manually create and wire thousands of objects. With Spring, you just **annotate classes correctly**, and the framework handles creation + wiring — letting you focus on **business logic**, not object plumbing.

Building a 3-layer example

DataService (Data Layer)

```
java
@Component
public class DataService {
    public List<Integer> getData() {
        return Arrays.asList(10, 20, 30, 40);
    }
}
```

BusinessService (Business Layer)

```
java
@Component
public class BusinessService {
    @Autowired
    private DataService dataService;

    public long calculateSum() {
        List<Integer> data =
            dataService.getData();

        return
            data.stream().reduce(Integer::sum).get();
    }
}
```

MyWebController (Web Layer)

```
java
@Component
public class MyWebController {
    @Autowired
    private BusinessService businessService;

    public long
    returnValueFromBusinessService() {
        return
            businessService.calculateSum();
    }
}
```

```
    }
}
```

Running it:

java

```
MyWebController controller =
context.getBean(MyWebController.class);

System.out.println(controller.returnValueFrom
mBusinessService());
```

Output: 100 (i.e., 10+20+30+40) — data flows Data -> Business -> Web layer automatically, with Spring wiring every dependency.

Dependency chain established

MyWebController --> BusinessService -->
DataService

(depends on) (depends on)

Each @Autowired field is auto-injected by Spring — no manual new statements anywhere.

Organizing into proper packages (realistic structure)

com.in28minutes.spring.learnspringframework
.enterprise.example.web ->
MyWebController

com.in28minutes.spring.learnspringframework
.enterprise.example.business ->
BusinessService

com.in28minutes.spring.learnspringframework
.enterprise.example.data -> DataService

Classes made public, moved to separate files/packages by layer (Web / Business / Data) — same as real production codebases. After moving, use **Organize Imports** to fix package references.

Key Takeaway — Answering Q3

Yes, Spring Framework adds massive value at scale.

- For 2-3 classes, manual wiring seems simpler.
- For **real enterprise apps with 1000s of classes across multiple layers**, manually creating and wiring every object is unmanageable.

- Spring lets you **just mark components** (@Component, @Service, @Repository) and dependencies (@Autowired) — the framework does all the creation, wiring, and lifecycle management.
- This lets developers focus on **business logic** instead of **object plumbing/configuration**.

9. Answer to Q4 — Switching Between Games (Handling Multiple Matching Beans)

Scenario

@Component is now present on **all three** game classes: MarioGame, PacmanGame, SuperContraGame — all implement GamingConsole.

Problem

Error: 3 matching beans found for GamingConsole.
Cannot autowire — ambiguous which one to inject.

Spring doesn't know which implementation GameRunner should get.

Solution: @Primary

Mark **one** implementation as the preferred/default bean when multiple candidates exist.

```
import
org.springframework.context.annotation.Primary;

@Component
@Primary
public class SuperContraGame implements
GamingConsole {
    ...
}
```

Result: Spring now injects SuperContraGame into GameRunner — even though 3 beans match GamingConsole, @Primary tells Spring "prefer this one."

Rule: Only ONE bean can be @Primary per type

If you mark **two** implementations @Primary (e.g., both MarioGame and SuperContraGame):

Error: No unique bean found — more than one 'primary' bean found among candidates.

Spring correctly rejects this — having 2 primaries defeats the purpose (still ambiguous).

Fix: Remove @Primary from all but one.

Key Takeaway

Situation	Behavior
1 bean matches	Works normally
0 beans match	Error — no bean found
2+ beans match, none @Primary	Error — ambiguous, unsatisfied dependency
2+ beans match, exactly 1 @Primary	Works — @Primary bean is injected
2+ beans match, 2+ marked @Primary	Error — still ambiguous

@Primary = "when multiple beans qualify for autowiring, prefer this one by default."

(This directly answers Q4: to switch games without touching GameRunner, just move the @Primary annotation to the desired game class.)

10. Answer to Q5 — How Is the Spring JAR Downloaded?

Answer: Maven

Maven is the **build tool** that downloads Spring JARs (and everything else) automatically.

Why not download JARs manually?

Spring Framework needs many other JARs to function — called **transitive dependencies**.

- Manually downloading Spring means also manually downloading **10, 20, even 100-200 other dependent JARs**

- Every time you upgrade Spring's version, you'd have to redo this manually
- Maven eliminates this pain entirely — it resolves and downloads **all transitive dependencies** for you

Where dependencies are defined: pom.xml

```
<dependencies>
    <dependency>

<groupId>org.springframework.boot</groupId>

        <artifactId>spring-boot-
starter</artifactId>

    </dependency>
    <dependency>

<groupId>org.springframework.boot</groupId>

        <artifactId>spring-boot-starter-
test</artifactId>

    </dependency>
</dependencies>
```

Just **two entries** in pom.xml trigger the download of dozens of JARs (visible under "Maven Dependencies" in the IDE).

Proof/Demo: Removing these two dependencies from pom.xml -> all Maven Dependencies disappear -> compilation errors everywhere. Undoing restores everything.

Identifying dependencies: Group ID + Artifact ID

Just like a Java class has a **class name + package**, every Maven dependency is identified by:

- **Group ID** (e.g., org.springframework.boot)
- **Artifact ID** (e.g., spring-boot-starter)

(Same concept used when we created our own project: Group com.in28minutes.spring, Artifact learn-spring-framework.)

Key Features of Maven

Feature	Description	# Question	Answer
Standard project structure	src/main/java, src/main/resources, src/test/java, pom.xml — consistent across ~99% of Java/Spring projects, so switching between projects is easy	How does 2 terminology relate?	Component, Dependency, Component Scan, DI, IoC, Beans, IoC Container, Autowiring — all mapped to actual code behavior
Dependency management	Handles both direct AND transitive dependencies automatically	3 Does Spring add real value?	Yes — critical at enterprise scale (1000s of classes, multiple layers) even though small examples look like "more code"
Version management	Change one version number in pom.xml -> all related dependencies update accordingly	4 How to switch games without editing GameRunner?	Use @Primary on the desired implementation when multiple beans match

Important Distinction: Maven Dependency vs Spring Dependency

Type	Level	Example
Maven Dependency	JAR/framework level (project needs a library)	Project needs spring-boot-starter JAR
Spring Dependency	Class/bean level (a bean needs another bean)	MyWebController needs BusinessService

Both use the word "Dependency" — always clarify which context is meant (Maven vs Spring) when discussing this term.

Summary — All 5 Questions Answered

# Question	Answer
What's 1 happening in background?	Spring scans @Component classes -> creates singleton beans -> autowires dependencies via constructor (visible in DEBUG logs)

Via **Maven** — resolves direct + transitive dependencies automatically from pom.xml (Group ID + Artifact ID)

11. Types of Dependency Injection in Spring

Spring provides **three types** of Dependency Injection:

1. Constructor-based Injection

@Autowired on a constructor. Used in GameRunner.

@Component

```
public class GameRunner {
    private GamingConsole game;

    @Autowired
    public GameRunner(GamingConsole game) {
        this.game = game;
    }
}
```

2. Setter-based Injection

@Autowired on a setter method.

@Component

```
public class BusinessService {  
    private DataService dataService;  
  
    @Autowired  
    public void  
    setDataService(DataService  
    dataService) {  
        this.dataService =  
        dataService;  
    }  
}
```

3. Field Injection

@Autowired directly on the field – **no constructor, no setter**. Uses **reflection** internally.

@Component

```
public class BusinessService {  
    @Autowired  
    private DataService dataService;  
}
```

(This was the original approach used in BusinessService.)

Which one to use?

Spring team recommends Constructor-based Injection.

Why:

- All dependencies are set **at the moment the object is created** (in one constructor call)
- If the constructor was called successfully, you can be **100% certain** all dependencies are properly wired — no partial/uninitialized state
- Especially useful when a class has many dependencies (e.g., 10 dependencies as constructor

parameters) — object is fully ready to use immediately after construction

Important detail

Even if @Autowired is placed on the **field** while a constructor **also exists**, Spring will still use **Constructor Injection** (constructor takes priority).

Quick Reference Table

Injection Type	Where @Autowired Goes	Mechanism	Recommended?
Constructor Injection	On the constructor	Direct, at object creation	Yes (best practice)
Setter Injection	On a setter method	Called after object creation	Sometimes (optional dependencies)
Field Injection	Directly on the field	Reflection	Not recommended (harder to test, hides dependencies)

12. Spring Framework Modules — Overview

Spring Framework is **not a monolith** — it's built as a **collection of modules**. You pick only what your application needs.

Key Modules

Module	Purpose
Core (IoC Container)	The foundation — manages beans, dependencies, ApplicationContext (what we've used throughout this course)
Test (Mock Objects / Spring MVC Test)	Writing unit tests for Spring applications

Module	Purpose
Transactions, JDBC, JPA	Talking to databases
Spring MVC	Building traditional Web Applications
Spring WebFlux	Building Reactive Web Applications (modern, non-blocking style)
Spring JMS	Integrating with other applications via message queues

Key Takeaway

- Spring's **modular architecture** = you choose only the modules relevant to your app's needs
- No need to include every module — e.g., a simple app might only need **Core**; a web app adds **Spring MVC**; a reactive app uses **WebFlux** instead
- This flexibility is a major reason Spring scales well from small apps to large enterprise systems

(This was a high-level "10,000 feet" overview — deeper dives into individual modules like MVC, JPA, etc. come later in the course.)

13. Spring Projects — Overview

While **Spring Framework** provides the core (IoC, DI, modules), **Spring Projects** are separate initiatives built on top to address evolving architecture trends (Web Apps -> REST APIs -> Microservices -> Cloud).

Why Spring stays relevant: Spring has been the dominant Java framework for 15+ years because it **keeps evolving** alongside changing architecture needs — this evolution happens via Spring Projects.

Key Spring Projects

Project	Purpose
Spring Boot	Most popular today — building REST APIs and Microservices quickly

Project	Purpose
Spring Cloud	Building cloud-native applications (designed to run natively on cloud platforms)
Spring Data	Unified way to integrate with both SQL (relational) and NoSQL databases
Spring Integration	Enables integration between different applications
Spring Security	Adds authentication, authorization , and other security features to Web Apps, REST APIs, Microservices

Key Takeaway

- **Spring Framework** = core engine (IoC/DI)
- **Spring Projects** = specialized extensions for modern needs (Boot, Cloud, Data, Security, Integration, etc.)
- Architecture trends evolved: **Web Apps -> REST APIs -> Microservices -> Cloud** — and Spring Projects evolved alongside each shift
- This is why Spring remains the go-to framework — modular core + continuously evolving ecosystem of projects

14. World Before Spring Boot — The Problems It Solves

New topic starting: **Spring Boot**. This step covers pain points **before** Spring Boot existed.

Problem 1: Dependency Management

To build a simple REST API, you had to manually manage in pom.xml:

- Spring Framework
- Spring MVC Framework
- JSON binding framework
- Logging framework
- For testing: Spring Test, Mockito, JUnit

Pain point: Manually tracking and matching **compatible versions** of every framework yourself.

Problem 2: Heavy Configuration (web.xml + Spring config)

Before running a web app, you needed to manually configure:

- DispatcherServlet (for Spring MVC)
- **Component Scan**
- **View Resolver** (for web apps)
- **Data Source** (for database apps)

Lots of boilerplate XML/Java config just to get started — before writing any actual business logic.

Problem 3: Non-Functional Requirements (NFRs)

Every production app needs:

- **Logging**
- **Error Handling**
- **Monitoring**
- **Metrics**

Before Spring Boot, developers had to **manually implement all of this from scratch**, every single project.

The Real Cost

- Setting up a new project could take **days**
- This setup had to be **repeated for every new project**
- Long-term: **countless hours** spent maintaining boilerplate configuration/NFR code instead of business logic

Key Takeaway

Before Spring Boot: **Dependency Management + Configuration + Manual NFR implementation** = slow, repetitive, error-prone project setup.

15. Building a Simple REST API with Spring Boot

(Note: this picks up assuming a Spring Boot project already exists — "creating a Spring Boot project" step wasn't included in what you sent. Let me know if you want that filled in separately.)

Goal

Expose a REST API at /courses that returns a list of courses (id, name, author).

Step 1: Create the Course model class

```
public class Course {  
  
    private long id;  
    private String name;  
    private String author;  
  
    public Course(long id, String name,  
String author) {  
  
        this.id = id;  
        this.name = name;  
        this.author = author;  
  
    }  
  
    public long getId()      { return id; }  
    public String getName() { return name; }  
    public String getAuthor() { return  
author; }  
  
    @Override  
    public String toString() {  
  
        return "Course [id=" + id + ",  
name=" + name + ", author=" + author + "];"  
    }  
}
```

(Generated via Eclipse: right-click -> Source -> Generate Constructor / Getters / toString — toString() conventionally placed at the end of the class.)

Step 2: Create CourseController

```

import
org.springframework.web.bind.annotation.Requ
estMapping;

import
org.springframework.web.bind.annotation.Rest
Controller;

import java.util.Arrays;

import java.util.List;

@RestController

public class CourseController {

    @RequestMapping("/courses")
    public List<Course> retrieveAllCourses()
{
    return Arrays.asList(
        new Course(1, "Learn AWS",
"in28minutes"),
        new Course(2, "Learn DevOps",
"in28minutes")
    );
}
}

```

Key Annotations

Annotation	Purpose
@RestController	Marks the class as a REST API controller — Spring handles converting return values (like List<Course>) to JSON automatically
@RequestMapping("/courses")	Maps the URL path /courses to this method

Running & Testing

1. Right-click LearnSpringBootApplication
-> **Run As -> Java Application** (must **stop** before restarting)

2. Visit: <http://localhost:8080/courses>
3. Returns raw JSON:

```

[{"id":1,"name":"Learn
AWS","author":"in28minutes"}, {"id":2,"name":"L
earn DevOps","author":"in28minutes"}]

```

Tip: Install a **JSON Formatter** browser extension (e.g., Chrome) to pretty-print the JSON response for readability.

Key Takeaway — The Power of Spring Boot

With Spring Boot, to build this REST API you needed **zero manual configuration**:

- No web.xml
- No manually defined Spring Beans
- No XML configuration
- No DispatcherServlet setup

You focused **only on business logic** (Course class + controller method). Everything else — server setup, JSON conversion, request routing — was handled automatically by Spring Boot.

16. The Goal of Spring Boot

Most Important Goal

Help you build **PRODUCTION-READY** applications **QUICKLY**.

Two key words: **QUICKLY** (faster development) + **PRODUCTION-READY** (all features needed to safely run in production).

Features that help build applications QUICKLY

Feature	What it does
Spring Initializr (start.spring.io)	Quickly generate a new Spring Boot project
Spring Boot Starter Projects	Quickly define/group dependencies for your project (no manual version-matching)
Spring Boot Auto Configuration	Automatically configures beans/settings based on what's in the classpath

Feature	What it does
	(which dependencies you added)
Spring Boot DevTools	Apply code changes without manually restarting the server

Features that make applications PRODUCTION-READY

Real production apps need:

- **Logging**
- **Different configuration per environment** (Dev, QA, Staging, Production)
- **Monitoring** (memory usage, health checks)
- **Metrics**

Feature	What it does
Default Logging	Spring Boot provides logging out of the box
Profiles	Manage different configurations for different environments (Dev/QA/Staging/Prod)
ConfigurationProperties	Externalize and manage configuration values
Spring Boot Actuator	Provides production monitoring — health checks, metrics, etc.

Key Takeaway

Spring Boot's value = **(Quick Development)** + **(Production Readiness)**, achieved through:

- **Quick dev:** Initializr, Starter Projects, Auto Configuration, DevTools

- **Production ready:** Default Logging, Profiles, ConfigurationProperties, Actuator

17. Spring Boot Starter Projects

The Problem

Building any feature requires **multiple frameworks**:

- **REST API:** Spring Framework, Spring MVC, Tomcat (server), JSON conversion
- **Unit Testing:** Spring Test, JUnit, Mockito

Manually adding each one + managing compatible versions = tedious.

The Solution: Starter Projects

Convenient dependency descriptors for a specific feature/use case — one dependency entry brings in a whole predefined, compatible set of libraries.

Example: pom.xml in our project

```
<dependency>

<groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>

<groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
</dependency>
```

What spring-boot-starter-web actually brings in

(Ctrl+Click / Cmd+Click on it in the IDE to inspect)

- **Spring Context** — launches the Spring container
- **spring-boot-starter-json** — JSON <-> Java object conversion
- **spring-boot-starter-tomcat** — embedded Tomcat server to run the app

- **Spring MVC** — framework for building REST APIs / web apps

All bundled under **one** starter dependency.

Common Spring Boot Starters

Starter	Use Case
spring-boot-starter-web	Web applications / REST APIs (brings Spring MVC, Tomcat, JSON)
spring-boot-starter-test	Unit testing (JUnit, Mockito, Spring Test)
spring-boot-starter-data-jpa	Talk to databases (Oracle, MySQL, etc.) using JPA
spring-boot-starter-jdbc	Talk to databases using JDBC
spring-boot-starter-security	Secure web apps / REST APIs

Key Takeaways

1. **Starter Projects = convenient dependency descriptors** — bundle everything needed for a specific feature/use case into one dependency
2. **Spring Boot provides many starters** — pick the right one(s) based on what your app needs (web, data, security, testing, etc.)

Important Note

Having the right dependencies in the classpath is **not enough by itself** — they also need to be **configured** to work together.

18. Spring Boot Auto Configuration

The Problem

Building a web app normally needs manual configuration: **Component Scan**, **DispatcherServlet**, **Data Source**, bean <-> JSON conversion, etc.

The Solution: Auto Configuration

Spring Boot **automatically configures** your application based on:

1. **What frameworks/classes are present in the Classpath** (determined by which Starter Projects you added)
2. **Your existing configuration** (defaults can be overridden if you provide your own config)

Where the logic lives

All Auto Configuration logic is defined in: **spring-boot-autoconfigure.jar**

Example package:
org.springframework.boot.autoconfigure.web -> handles all Web/REST API related auto configuration.

Inspecting Auto Configuration via DEBUG logs

```
# application.properties
```

```
logging.level.org.springframework=debug
```

Restart the app (stop -> start) -> look for **"CONDITIONS EVALUATION REPORT"** in logs:

Match Type	Meaning
Positive matches	Things that WERE auto-configured (e.g., DispatcherServlet, Tomcat, Error page)
Negative matches	Things that were NOT auto-configured

Key Auto Configuration classes (inspect via Ctrl+Shift+T / Cmd+Shift+T)

1. DispatcherServletAutoConfiguration

- Auto-enabled when: app is a Web App/REST API AND DispatcherServlet class is on the classpath
- Since we included spring-boot-starter-web, DispatcherServlet is present -> gets auto-configured

2. ErrorMvcAutoConfiguration

- Configures a **default error page** (the "Whitelabel Error Page")
- Demo: hitting an unmapped URL like /courses/10 -> returns a proper 404 Not

Found Whitelabel Error Page automatically — no manual error handling written

3. JacksonHttpMessageConvertersConfiguration

- Auto-configures **JSON conversion** (via the **Jackson** library)
- This is why returning a List<Course> from a controller method automatically converts to JSON in the response — no manual converter code needed

What Spring Boot Starter Web Auto-Configures (summary)

Auto-Configured Component	Purpose
DispatcherServlet	Front controller for handling web requests
Embedded Tomcat	Runs the application without needing an external server
Default Error Pages	Handles unmapped URLs / errors gracefully (404, etc.)
JSON Message Converter (Jackson)	Converts Java Beans <-> JSON automatically

Key Takeaway

Starter Projects bring dependencies -> Auto Configuration examines those dependencies (JARs/classes) on the classpath -> automatically configures beans/settings accordingly.

This is the core mechanism behind why adding just spring-boot-starter-web gives you a fully working REST API server with zero manual XML/Java configuration.

Improves **developer productivity**. Question it answers: *"Why manually restart the server for every code change?"*

Adding DevTools

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-devtools</artifactId>
</dependency>
```

Add to pom.xml -> wait for Maven to download -> then **Stop and Start** the server once (only needed this one time for the initial setup).

How It Works

Once DevTools is added:

- Any change to a **Java file, property file, or view file** -> automatically triggers a **server restart** for you
- No manual stop/start needed for these file types

Demo Flow

1. Added a new course (Learn Azure, id 3) to CourseController
2. On save -> automatic restart triggered in console
3. Refresh browser -> new course appears in response
4. (Even a syntax error triggers an automatic restart attempt — fix the error, save again, auto-restart picks up the fix)

Important Exception

Changes to pom.xml are NOT picked up automatically by DevTools.

If you add/remove a dependency in pom.xml, you must **manually Stop and Start** the application yourself.

Summary Table

File Type Changed	DevTools Auto-Restarts?
Java files (.java)	Yes

19. Spring Boot DevTools

Purpose

File Type Changed	DevTools Auto-Restarts?
Property files (application.properties)	Yes
View/template files	Yes
pom.xml (dependencies)	No — manual restart required

Key Takeaway

Spring Boot DevTools = **faster feedback loop** during development — save your code, and the app restarts itself, so you can immediately test changes without manually stopping/starting the server every time.

20. Managing Configuration Using Profiles

(First production-readiness feature)

The Problem

Applications run in **different environments**: Dev, QA, Stage, Production. Same code, but each environment needs **different configuration** (e.g., different database, different web service URL, different logging level).

The Solution: Spring Profiles

Create **separate property files per environment**, then activate the one you need.

Naming Convention

application.properties	-> default (used when no profile is active)
application-dev.properties	-> dev environment
application-prod.properties	-> production environment
application-qa.properties	-> qa environment

Example Setup

application.properties (default)
properties

logging.level.org.springframework=debug

application-dev.properties

properties

logging.level.org.springframework=trace

application-prod.properties

properties

logging.level.org.springframework=info

Activating a Profile

properties

in application.properties

spring.profiles.active=prod

How merging works:

- Default (application.properties) values + active profile's values are **merged**
- If a property exists in **both**, the **active profile's value wins** (higher priority)
- Example: default has debug, but prod profile has info -> since prod is active, **info wins**

Switching profiles

Change spring.profiles.active=dev -> now application-dev.properties (trace) takes priority instead.

Logging Levels (Priority Order — Low to High Verbosity Restriction)

Level What Gets Printed

trace Everything (trace + debug + info + warn + error)

debug debug + info + warn + error

info info + warn + error

warn warn + error

error only error

off Nothing — logging disabled

Rule: Setting a level prints that level **and everything below it** in the list (i.e., less verbose levels). trace = most verbose (prints everything), off = least (prints nothing).

Key Takeaway

- **Profiles** = environment-specific configuration files (application-{profile}.properties)
- **spring.profiles.active** = controls which profile's settings are merged/prioritized over the defaults
- Enables the **same codebase** to behave differently (different DB, different logging, etc.) depending on which environment it's deployed to — without changing code

21. @ConfigurationProperties — Managing Complex Configuration

The Problem

Real apps need many config values, e.g., for a currency service:

currency-service.url

currency-service.username

currency-service.key

Hardcoding or manually reading dozens of properties one by one doesn't scale.

The Solution: @ConfigurationProperties

Recommended Spring Boot approach for managing **complex/grouped application configuration**.

Step 1: Create the configuration class

```
import
org.springframework.boot.context.properties.
ConfigurationProperties;

import
org.springframework.stereotype.Component;

@Component

@ConfigurationProperties(prefix = "currency-
service")

public class CurrencyServiceConfiguration {

    private String url;

    private String username;
```

```
private String key;

// Getters and Setters (generated)

public String getUrl() { return url; }

public void setUrl(String url) {
this.url = url; }

public String getUsername() { return
username; }

public void setUsername(String username)
{ this.username = username; }

public String getKey() { return key; }

public void setKey(String key) {
this.key = key; }

}
```

Key points:

- @ConfigurationProperties(prefix = "currency-service") -> maps properties with that prefix
- @Component -> lets Spring manage this as a bean
- **Setters are required** (unlike constructor injection used elsewhere) since Spring populates fields via setters here

Step 2: Define values in application.properties

```
currency-
service.url=http://default.in28minutes.com

currency-service.username=defaultusername

currency-service.key=defaultkey
```

Spring automatically **binds** these properties to the matching fields (url, username, key) based on the prefix.

Step 3: Verify via a test controller

```
@RestController

public class CurrencyConfigurationController
{

    @Autowired

    private CurrencyServiceConfiguration
configuration;
```

```

    @RequestMapping("/currency-configuration")

    public CurrencyServiceConfiguration
retrieveConfiguration() {

        return configuration;

    }
}

```

Visit: <http://localhost:8080/currency-configuration> -> returns the configured values as JSON.

Combining with Profiles

Different environments can override these values:

application-dev.properties

```

currency-service.url=dev.in28minutes.com
currency-service.username=devusername
currency-service.key=devkey

```

When `spring.profiles.active=dev` -> the `/currency-configuration` endpoint now returns the **dev-specific values** instead of the defaults (profile values take priority, as learned in Section 20).

Key Takeaway

- `@ConfigurationProperties` = centralize and structure **all related config values into one class**, instead of reading raw string properties manually everywhere
- Can scale to **hundreds of properties** in real applications — acts as a **single source of truth** for a feature's configuration
- **@ConfigurationProperties + Profiles** together = powerful way to **externalize configuration** per environment, without touching code

Quick Reference

Concept	Role
<code>@ConfigurationProperties(prefix=...)</code>	Binds a group of properties (by prefix) to a Java

Concept	Role
	class's fields
<code>@Component</code>	Registers the config class as a Spring-managed bean
<code>application.properties</code>	Default values
<code>application-{profile}.properties</code>	Environment-specific overrides
<code>@Autowired</code>	Inject the configuration bean wherever needed

22. Embedded Servers — Simplifying Deployment

The Old Approach: WAR Deployment

Traditional deployment required 3 separate steps:

1. Install **Java**
2. Install a **Web/Application Server** (Tomcat, WebLogic, WebSphere)
3. Deploy the application as a **WAR file** onto that server

More setup steps = more complexity, especially when deploying frequently across Dev, QA, Staging, Production (common in Agile with multiple deployments/day).

The Spring Boot Approach: Embedded Server

The server (e.g., **Tomcat**) is **bundled inside your JAR file itself**.

Result: All you need is **Java installed** -> then just run the JAR.

Building and Running the JAR

Step 1: Build the JAR

Right-click project → Run As → Maven Build

Goals: clean install

This runs mvn clean install and produces a JAR, e.g.:

learn-spring-boot-0.0.1-SNAPSHOT.jar

Step 2: Run the JAR

cd <target-folder>

java -jar learn-spring-boot-0.0.1-SNAPSHOT.jar

Note: Requires Java 17+ (Spring Boot 3 requirement) — any newer Java version works too. Also, stop any already-running instance first, or you'll get a "port 8080 already in use" error.

Once running, hitting localhost:8080/courses works exactly as before — no separate web server installation needed.

Why This Works

spring-boot-starter-web automatically brings in spring-boot-starter-tomcat as a dependency — Tomcat is **embedded directly into your JAR**. That's why only Java is required to run it.

Famous Quote

"**Make JAR not WAR**" — Josh Long, Spring/Spring Boot Developer Advocate

Supported Embedded Servers

Server	Notes
Tomcat	Default (comes via spring-boot-starter-tomcat)
Jetty	Alternative option
Undertow	Alternative option

Key Takeaway

Embedded Servers = simplified deployment.

No separate server installation — just Java + your self-contained JAR file. This is a core reason Spring Boot apps are easy to deploy consistently across environments and enables fast, frequent deployments (CI/CD friendly).

23. Spring Boot Actuator — Monitoring in Production

Purpose

Helps you **monitor and manage** your application in production by exposing useful runtime information through HTTP endpoints.

Adding Actuator

```
<dependency>
```

```
<groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Default Behavior

By default, only the **health** endpoint is exposed:

```
localhost:8080/actuator ->
shows self link + health link

localhost:8080/actuator/health ->
{"status": "UP"}
```

Exposing All Endpoints

```
management.endpoints.web.exposure.include=*
```

This exposes **all** Actuator endpoints (restart app to pick up change).

Important Actuator Endpoints

Endpoint	Shows
/actuator/health	Whether the application is up and running (default, always available)
/actuator/beans	All Spring beans loaded (e.g., courseController, DispatcherServlet, auto-configured beans) — useful to verify if something got auto-configured

Endpoint	Shows	
/actuator/configprops	All configurable properties (including custom @ConfigurationProperties classes like currency-service.*) and their current values , including which profile they came from	Enabling many endpoints = more data gathering = more CPU and memory usage . Best practice: Explicitly configure only the endpoints you actually need: management.endpoints.web.exposure.include=health,metrics Instead of exposing everything (*) in production.
/actuator/env	Environment details — port, Java version, encoding, classpath, JVM/Tomcat info	Key Takeaway Spring Boot Actuator = production monitoring toolkit — gives visibility into: • Application health • Loaded beans/configuration • Environment/runtime details • Live metrics (requests, JVM, disk, etc.)
/actuator/metrics	List of available metrics (app start time, disk space, executor pool, JVM stats, etc.)	Combined with Logging, Profiles, and ConfigurationProperties , this completes the "Production-Ready" side of Spring Boot's goal (Section 16).
/actuator/metrics/http.server.requests	Specific metric — request COUNT, TOTAL_TIME, MAX time to handle requests. Increases as you hit endpoints, showing live tracking	
/actuator/mappings	All configured request mappings (URL routes) in the application	

Example: Checking Configuration Source

Visiting /actuator/configprops confirmed that currency-service.url value (dev.in28minutes.com) was being picked up from **application-dev.properties** — ties directly back to the Profiles concept (Section 20-21).

Important Caution

24. Spring Framework vs Spring MVC vs Spring Boot

Important clarification: **these are not competitors** — Spring Boot is a **wrapper** that makes it easier to use Spring Framework and Spring MVC.

Spring Framework (Core)

Focus: Dependency Injection (DI)

- Define dependencies with annotations: @Component, @Service, etc.
- Identify components via **Component Scan**
- **Autowire** dependencies together

Limitation: DI alone isn't enough to build full applications — you also need other frameworks (e.g., Hibernate/JPA for databases, JUnit/Mockito for testing).

Role of Spring Modules & Spring Projects:

Extend the Spring ecosystem — provide

integration with these other frameworks (databases, testing, security, etc.)

Spring MVC (a Spring Module)

Focus: Simplifying building of **Web Applications and REST APIs** only.

- Predecessor: **Struts** — was complex, had a lot of tight coupling
- Spring MVC provided a **better, more loosely-coupled approach**
- Key annotations: @Controller, @RestController, @RequestMapping (used in our CourseController earlier)

Spring Boot (a Spring Project)

Goal: Help build **PRODUCTION-READY applications QUICKLY** (recap from Section 16)

Problem it solves: Even with Spring + Spring MVC, you still needed lots of manual config in pom.xml, web.xml, applicationContext.xml.

Key features:

- **Starter Projects** -> bundle needed dependencies (e.g., spring-boot-starter-web for web apps)
- **Auto Configuration** -> eliminates manual Spring/Spring MVC setup, configures automatically based on classpath
- **Actuator** -> monitoring
- **Embedded Servers** -> simplified deployment
- **Default Logging & Error Handling**
- **Profiles + ConfigurationProperties** -> simplified configuration management

Summary Comparison Table

	Spring Framework	Spring MVC	Spring Boot
Type	Core framework	Spring Module	Spring Project

	Spring Framework	Spring MVC	Spring Boot
Core Focus	Dependency Injection	Web apps / REST APIs	Fast + production-ready app development
Key Concepts	@Component, @ComponentScan, Autowiring	@Controller, @RestController, @RequestMapping	Starter Projects, Auto Configuration, Actuator, Embedded Servers, Profiles
Relationship	Foundation everything else builds on	Built on top of Spring Framework	Wraps Spring + Spring MVC to simplify setup

Key Takeaway

Spring Framework = DI engine → **Spring MVC** = web layer built on top of it → **Spring Boot** = convenience wrapper that eliminates manual configuration for both, while adding production-readiness features.

None of these compete — they form a **layered ecosystem**, each building on the one before it.

25. Building Your First Web Application — "Hello World"

(New sub-section: My First Web App)

Setup

Create SayHelloController in package: com.in28minutes.springboot.myfirstwebapp.hello (must be a **sub-package** of the main application package, myfirstwebapp)

Goal

When user hits /say-hello, return: "Hello! What are you learning today?"

Building it up (step by step, showing each error and fix)

Attempt 1 — Basic method (fails)

```
public class SayHelloController {  
    @RequestMapping("say-hello")  
    public String sayHello() {  
        return "Hello! What are you  
learning today?";  
    }  
}
```

Result: **404 Whitelabel Error Page — Not Found** Reason: Spring doesn't know this class is a bean it should manage.

Attempt 2 — Add @Controller (still fails)

```
@Controller  
public class SayHelloController {  
    @RequestMapping("say-hello")  
    public String sayHello() {  
        return "Hello! What are you learning  
today?";  
    }  
}
```

Result: Still an error. Reason: By default, Spring MVC treats a returned String as the **name of a View** to render (like a JSP/HTML page), not as raw text. It looks for a view called "Hello! What are..." and fails to find it.

Attempt 3 — Add @ResponseBody (works)

```
import  
org.springframework.stereotype.Controller;  
  
import  
org.springframework.web.bind.annotation.Requ  
estMapping;  
  
import  
org.springframework.web.bind.annotation.Resp  
onseBody;  
  
@Controller  
public class SayHelloController {
```

```
    @RequestMapping("say-hello")  
    @ResponseBody  
    public String sayHello() {  
        return "Hello! What are you learning  
today?";  
    }  
}
```

Result: Visiting <http://localhost:8080/say-hello> returns the string directly in the browser.

Key Annotations Explained

Annotation	Purpose
@Controller	Tells Spring: "this class handles web requests" — makes it a Spring-managed bean
@RequestMapping("say-hello")	Maps the URL path /say-hello to this method
@ResponseBody	Returns the method's return value as-is (raw text/data) to the browser, instead of treating it as a view name to resolve

Important Notes

- **Run the app from the main class** (the one with main() method, e.g., MyfirstwebappApplication) — you **cannot** "Run As Java Application" from SayHelloController since it has no main() method.
- Default port: **8080** (check logs to confirm) -> full URL: <http://localhost:8080/say-hello>
- If it doesn't work, double-check:
 - Correct imports (@Controller, @RequestMapping, @ResponseBody from org.springframework...)

- URL in browser matches the `@RequestMapping` value exactly
- Controller class is in a **sub-package** of the main application package

Key Takeaway

To build a simple Spring MVC endpoint that returns raw text, you need **all three**:

1. `@Controller` → registers it as a request-handling bean
2. `@RequestMapping` → maps a URL to the method
3. `@ResponseBody` → sends the return value directly, not as a view name

(Note: `@RestController` used earlier for the REST API combines `@Controller` + `@ResponseBody` automatically — that's why it didn't need `@ResponseBody` separately.)

26. Returning HTML — Why Hardcoding Doesn't Scale

Goal

Return actual HTML content from a controller method instead of plain text.

Attempt: Hardcoding HTML using `StringBuffer`

`@Controller`

```
public class SayHelloController {

    @RequestMapping("say-hello-html")
    @ResponseBody
    public String sayHelloHtml() {
        StringBuffer sb = new
StringBuffer();

        sb.append("<html>");
        sb.append("<head>");
        sb.append("<title>My First HTML
Page</title>");
        sb.append("</head>");
        sb.append("<body>My first html page
with body</body>");
        sb.append("</html>");
    }
}
```

```
        return sb.toString();
    }
}
```

Why `StringBuffer`/`StringBuilder` instead of plain String concatenation?

Repeated String concatenation (`+=`) is inefficient — creates a new String object each time.

`StringBuffer`/`StringBuilder` is the efficient way to build up content piece by piece.

Testing

`http://localhost:8080/say-hello-html` → browser renders:

My first HTML page with body (Tab title: "My First HTML Page")

The Problem

Even this **tiny** HTML snippet became messy and hard to manage:

- Formatting is lost inside `.append()` calls
- Every small change means editing Java code and restarting
- **Doesn't scale** — imagine a real 200-500 line HTML page built this way

Key Takeaway

Hardcoding HTML inside Java controller methods (via `String`/`StringBuffer`) is **not maintainable** for real applications.

Better approach (coming next): Use **View templates** (like **JSP**, **Thymeleaf**, etc.) to separate HTML from Java code — this is the standard, scalable way to build web pages in Spring MVC.

(Next step: Building your first "View")

27. Building Your First View — JSP

The Problem

Hardcoding HTML in Java (previous step) doesn't scale. **Views** separate HTML from Java code. **JSP (Java Server Pages)** is one of the earliest and most popular view technologies.

Step 1: Create the required folder structure

JSPs **must** go in this exact path:

```
src/main/resources/META-INF/resources/WEB-INF/jsp/
```

Create it folder-by-folder: META-INF -> resources -> WEB-INF -> jsp

Step 2: Create the JSP file

```
src/main/resources/META-INF/resources/WEB-INF/jsp/sayHello.jsp
```

```
<html>
<head>
    <title>My First HTML Page - JSP</title>
</head>
<body>
    My first html page with body - JSP
</body>
</html>
```

Editing HTML directly in a .jsp file is far easier than building it via StringBuffer in Java.

Step 3: Configure View Resolver (prefix/suffix) in application.properties

```
spring.mvc.view.prefix=/WEB-INF/jsp/
```

```
spring.mvc.view.suffix=.jsp
```

(Spring MVC already knows the base path up to src/main/resources/META-INF/resources/ — you only specify from /WEB-INF/jsp/ onward.)

Why prefix/suffix? Instead of hardcoding full paths for every view, you configure the constant part once. The controller just returns the **variable part** (the view name, e.g., sayHello), and Spring MVC assembles:

```
prefix + viewName + suffix = /WEB-INF/jsp/sayHello.jsp
```

Step 4: Update the Controller — return the VIEW NAME, not text

```
@Controller
public class SayHelloController {

    @RequestMapping("say-hello-jsp")
    public String sayHelloJsp() {
        return "sayHello"; // name of the
        view, NOT the content
    }
}
```

```
}
```

Critical: Do NOT use @ResponseBody here.

- @ResponseBody -> returns the string **directly as the response body** (raw text, no view lookup)
- Without @ResponseBody -> Spring MVC treats the returned string as a **view name** and resolves it via the View Resolver -> redirects to the JSP

This is the opposite of what we needed in Section 25 (there we wanted raw text, so @ResponseBody was required).

Step 5: Debugging a missing dependency

Even with everything configured correctly, JSP rendering initially failed. Root cause: **missing JAR** to support running JSPs on the embedded Tomcat server.

Fix — add to pom.xml:

```
<dependency>

<groupId>org.apache.tomcat.embed</groupId>

    <artifactId>tomcat-embed-jasper</artifactId>

    <scope>provided</scope>
</dependency>
```

(scope=provided because Tomcat already provides this at runtime.)

Important: Changes to pom.xml require a **manual Stop + Start** (DevTools does NOT auto-restart for pom.xml changes — recall Section 19).

Debugging Tip Used

Enabling DEBUG logging helped trace the issue:

```
logging.level.org.springframework=debug
```

Logs showed the request was correctly resolving to /WEB-INF/jsp/sayHello.jsp — confirming the View Resolver config was right, and narrowing the problem down to the missing dependency.

Final Result

http://localhost:8080/say-hello-jsp -> renders the JSP content, confirmed as coming from the actual .jsp file (not hardcoded Java).

Full Request Flow (Summary)

Browser request: /say-hello-jsp

↓

SayHelloController.sayHelloJsp() is invoked (via @RequestMapping)

↓

Method returns view name: "sayHello"

↓

View Resolver applies prefix + name + suffix:

/WEB-INF/jsp/ + sayHello + .jsp

↓

Renders: /WEB-INF/jsp/sayHello.jsp

Key Takeaway Table

Concept	Role
View (JSP)	Separates HTML/presentation from Java/business logic
Controller method return value (no @ResponseBody)	Treated as a view name , not raw content
spring.mvc.view.prefix / suffix	Auto-assembles full view path from just the view name
View Resolver	Spring MVC component that maps a view name to an actual file using prefix/suffix
tomcat-embed-jasper	Required dependency to render JSPs on embedded Tomcat
@ResponseBody	Returns raw content directly — skips view resolution entirely

28. Exercise — Creating a Login Page (JSP + Controller)

Task

When user visits /login, show content from login.jsp, using a new LoginController in package:

```
com.in28minutes.springboot.myfirstwebapp.login
```

Step 1: Create LoginController

```
package
com.in28minutes.springboot.myfirstwebapp.login;
```

```
import
org.springframework.stereotype.Controller;

import
org.springframework.web.bind.annotation.RequestMapping;
```

```
@Controller

public class LoginController {

    @RequestMapping("login")
    public String gotoLoginPage() {
        return "login";
    }
}
```

Common mistake to watch for: Forgetting @Controller -> results in **404 "Resource not found"** because Spring doesn't know this class handles web requests (same lesson as Section 25).

Step 2: Create login.jsp

Location: src/main/resources/META-INF/resources/WEB-INF/jsp/login.jsp

```
<html>
<head>

    <title>Login Page</title>
</head>
<body>

    Welcome to the login page!
```

</body>

</html>

Testing

http://localhost:8080/login -> displays
"Welcome to the login page!"

Key Insight — Why This Was Fast

All the **one-time setup** (adding tomcat-embed-jasper to pom.xml, configuring spring.mvc.view.prefix/suffix in application.properties) was already done in Section 27.

From now on, adding any new page only requires:

1. A new **Controller** (with @Controller + @RequestMapping)
2. A new **JSP** in the WEB-INF/jsp folder
3. Return the view name matching the JSP filename (without .jsp)

No further pom.xml or application.properties changes needed for additional views.

Recap — Full Pattern for Adding a New Page

1. Create Controller in its own package (e.g., .login, .hello)
2. @Controller + @RequestMapping("url-path")
3. Method returns view name (String) — matches JSP filename
4. Create matching .jsp file in WEB-INF/jsp/
(Exercise suggestion from video: try creating another Controller + JSP combo for more practice — e.g., a "welcome" page.)

29. HTTP Basics — How the Web Works (Request/Response Flow)

Viewing what the browser actually receives

Using **right-click -> View Page Source**:

- /say-hello -> plain text response ("Hello! What are you learning today?")

- /say-hello-html -> actual **HTML** source (from the hardcoded HTML in the controller)
- /say-hello-jsp -> also **HTML** source (rendered from the JSP)

Key insight: The browser receives raw HTML/text and renders it visually — you can inspect the true response via "View Page Source."

Using HTML tags in JSP

JSPs can contain any standard HTML tags:

<h1>Heading 1</h1>

<h2>Heading 2</h2>

These render exactly like in any HTML page.

Inspecting Network Traffic (DevTools)

Right-click -> Inspect -> Network tab ->
refresh page -> select the request (e.g., say-hello-jsp)

Key details visible for each request:

Field	Example Value	Meaning
Request URL	http://localhost:8080/say-hello-jsp	The exact URL requested
Request Method	GET	The HTTP method used (others: POST, etc. — covered later)
Status Code	200	Response status — indicates success or failure

HTTP Status Codes

Code Meaning

200 Success — request handled properly

Code Meaning

404 Not Found — no controller/mapping exists for that URL

Demo: Hitting a non-existent URL (say-hello-jsp-1) -> Whitelabel Error Page -> Network tab shows **Status Code 404**.

Request Headers vs Response Headers vs Response Body

- **Response tab** in DevTools -> shows the exact content sent back to the browser
- **Request Headers / Response Headers** -> additional metadata sent with the request/response (not deeply covered here yet)

The Core Flow (Big Picture)

Browser

| sends



HttpRequest ———> Spring Boot Web Application

| handles request



Browser <———— HttpServletResponse
(typically HTML)

1. **Browser** sends an **HttpRequest** (URL, method, headers)
2. **Server** (our Spring Boot app) receives and **handles** the request (via Controller)
3. Server sends back an **HttpResponse** (usually HTML, could be text/JSON) — includes a **Status Code**
4. Browser **renders** the response

Key Takeaway

Every browser interaction = an **HTTP request-response cycle**. Understanding this flow (Request -> Handling -> Response, with Status Codes) is fundamental to debugging any web

application — checking Network tab / Status Codes is one of the first things to do when something doesn't work.

30. Request Parameters and the Model

The Goal

Turn `http://localhost:8080/login?name=Yash` into a personalized response: **"Welcome to login page, Yash!"**

Step 1: Passing data via Query Parameter

URL format:

`localhost:8080/login?name=Yash`

The part after `?` is a **query parameter** — a simple way to pass data to a URL.

Step 2: Capture it in the Controller using `@RequestParam`

```
import
org.springframework.web.bind.annotation.Req
uestParam;
```

`@Controller`

```
public class LoginController {
```

```
    @RequestMapping("login")
```

```
    public String
```

```
    gotoLoginPage(@RequestParam String name) {
```

```
        System.out.println("Request param is " + name); // NOT recommended for production
```

```
        return "login";
```

```
    }
```

```
}
```

@RequestParam — binds a method parameter to a web request parameter. The parameter name (name) must match the query parameter name in the URL.

(Note: `System.out.println` is used here only as a quick way to verify — proper logging practices covered in the next step.)

Step 3: Passing data to the JSP via Model

The Controller and the View (JSP) are separate — you can't directly access Java variables

inside a JSP. To pass data across, use **ModelMap**.

```
import org.springframework.ui.ModelMap;
```

```
@RequestMapping("login")
public String gotoLoginPage(@RequestParam
String name, ModelMap model) {
    model.put("name", name);
    return "login";
}
```

ModelMap — a container the Controller uses to pass data (attributes) to the View.

model.put("name", name) stores the value under the key "name".

Step 4: Access the value in the JSP using Expression Language (EL)

```
<body>
    Welcome to login page ${name}!
</body>
```

Expression Language syntax:

`${variableName}` — the name inside `${}` must match the key used in `model.put(...)`.

Result

localhost:8080/login?name=Yash ->
"Welcome to login page Yash!"

localhost:8080/login?name=John ->
"Welcome to login page John!"

Full Flow (Summary)

Browser sends: /login?name=Yash

↓

@RequestParam String name -> captures
"Yash" from query param

↓

model.put("name", name) -> stores it for the
view

↓

JSP: `${name}` -> renders "Yash" in the
HTML output

Key Concepts Recap

Concept	Role
Query Parameter	Data passed in the URL after ?key=value
@RequestParam	Binds a query parameter to a Controller method parameter
ModelMap	Carries data from Controller → View
<code>\${name}</code> (Expression Language)	Retrieves a model attribute inside a JSP

Learning Tip (from the instructor)

Get a working example first, then **learn by breaking things** — modify the Controller and JSP, see what happens, and understand the "why" behind each error.

Exercise suggestion: Try changing parameter names, adding a second @RequestParam, or breaking the model key/EL variable mismatch intentionally to see the resulting error.

31. Logging Best Practices

Why Logging Matters

When debugging problems in production, **logs are often the only thing you have**. Choosing what and how to log is a critical skill.

Configuring Log Levels in application.properties

```
logging.level.org.springframework=debug
```

- **debug** -> prints a LOT of information (useful in development, causes performance impact in production)
- **info** -> prints much less (typical for production)

You can target specific packages too:

```
logging.level.com.in28minutes.springboot.myfi
rstwebapp=debug
```

This sets debug level **only for your application's own package** — everything else (like org.springframework) can remain at a different (e.g., info) level.

Problem with System.out.println

- Cannot control its level
- Always prints regardless of configuration
- **Not recommended** for real applications

The Correct Way: Use a Logger

```
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;

@Controller

public class LoginController {

    private Logger logger =
LoggerFactory.getLogger(getClass());

    @RequestMapping("login")

    public String
gotoLoginPage(@RequestParam String name,
ModelMap model) {

        logger.debug("Request param is {}",
name);

        model.put("name", name);

        return "login";

    }
}
```

Key points:

- `LoggerFactory.getLogger(getClass())` — factory pattern to create a Logger instance for the current class
- `{}` — placeholder syntax (instead of string concatenation) — gets replaced by the argument
- Uses **SLF4J** (`org.slf4j.Logger`) — make sure you import the correct one (not `java.util.logging.Logger`)

Logging Levels & Methods

Method	Level	When to Use
<code>logger.debug(...)</code>	DEBUG	Detailed dev/debugging info — never shown in

Method	Level	When to Use
<code>logger.info(...)</code>	INFO	production (typically) General important application events — typical production level
<code>logger.warn(...)</code>	WARN	Potential issues, not errors
(also: <code>logger.error(...)</code>)	ERROR	Actual errors/exceptions

Rule (recap from Section 20): Setting a level prints that level **and everything below it** in severity (e.g., info prints info + warn + error; debug prints debug + info + warn + error).

Demonstrated Behavior

- With `logging.level...=debug` -> `logger.debug(...)` message **appears** in logs
- Change to `logging.level...=info` -> the same `logger.debug(...)` message **disappears** — only info/warn level logs show
- `System.out.println` **always prints regardless of configured level** — proving why it's a bad practice

Where Logging Comes From

Spring Boot uses **spring-boot-starter-logging** — a **transitive dependency** brought in automatically by `spring-boot-starter-web` (visible in `pom.xml` → Dependency Hierarchy tab).

Default logging framework: Logback with **SLF4J** (the abstraction layer/API you code against).

Cleanup Step

Before moving forward, the video removes:

- Commented-out code
- `System.out.println` statements
- The temporary Logger statements

- Unused imports (Ctrl+D / Cmd+D to delete full lines)

Resulting in a clean LoginController as the new baseline for future steps.

Key Takeaway

Concept	Summary
Never use <code>System.out.println</code> in real apps	No level control, always prints
Use SLF4J Logger	<code>LoggerFactory.getLogger(getClass())</code>
Configure levels per package in <code>application.properties</code>	Fine-grained control over verbosity
<code>debug</code> = dev only, <code>info</code> = production default	Balances detail vs performance
<code>spring-boot-starter-web</code> already brings logging	Via <code>spring-boot-starter-logging</code> (Logback + SLF4J)

32. Spring MVC Architecture — DispatcherServlet Deep Dive

A Bit of History

Model 1 Architecture

- **Everything** (view logic, flow logic, database logic) lived inside JSPs
- No Controllers at all
- Result: **zero separation of concerns**, very hard to maintain

Model 2 Architecture

- Introduced clear separation:
 - **Model** = the data used to generate the view
 - **View** = displays information to the user

- **Controller/Servlet** = controls the flow

- Better maintainability, but... common features (like Authentication) had to be duplicated across every Servlet

Model 2 with Front Controller

- Introduced a **single central Controller** ("Front Controller") that receives **all** requests
- Common features (security, authentication, authorization) implemented **once**, in this front controller
- Front Controller then forwards to the right Controller/Servlet and View

Spring MVC's Implementation: DispatcherServlet

DispatcherServlet = Spring MVC's implementation of the Front Controller pattern.

Every single request (whether `/login`, `/say-hello`, `/say-hello-jsp`) is **first received by DispatcherServlet**.

The Full Request Flow

1. Browser sends request → e.g., `GET /login`
↓
2. DispatcherServlet (Front Controller) receives it
↓
3. DispatcherServlet identifies the right Controller + method
(e.g., `LoginController.gotoLoginPage()`)
↓
4. Controller method executes:
 - Puts data into the Model (`model.put(...)`)
 - Returns the View NAME (e.g., `"login"`)
 ↓
5. DispatcherServlet asks the ViewResolver:

"What is the actual view for name 'login'?"

↓

6. ViewResolver applies prefix + name + suffix:

/WEB-INF/jsp/ + login + .jsp

= /WEB-INF/jsp/login.jsp

↓

7. DispatcherServlet executes/renderers that JSP

↓

8. HTTP Response sent back to the browser

Key Components & Their Roles

Component	Role
DispatcherServlet	Front Controller — receives ALL requests, orchestrates the entire flow
Controller (e.g., LoginController)	Handles specific URL, executes business logic, populates Model, returns view name
Model (ModelMap)	Carries data from Controller to View
View Resolver	Maps a view name (e.g., "login") to the actual file path (e.g., /WEB-INF/jsp/login.jsp) using configured prefix/suffix
View (JSP)	Renders the final HTML shown to the user

Concrete Example Walkthrough

URL: localhost:8080/login

↓

DispatcherServlet identifies:
LoginController.gotoLoginPage()

↓

gotoLoginPage() executes:

- model.put("name", ...)

- returns "login"

↓

ViewResolver maps "login" → WEB-INF/jsp/login.jsp

↓

DispatcherServlet renders login.jsp

↓

Rendered HTML sent back as HTTP Response

Key Takeaway

DispatcherServlet is the single entry point for every request in a Spring MVC application. It coordinates between **Controller** (business logic), **Model** (data), and **View Resolver + View** (presentation) to produce the final response — this is the concrete mechanism behind the "Front Controller" architectural pattern.

This ties together everything learned so far:

- Section 25 (@Controller, @RequestMapping)
- Section 27 (View Resolver prefix/suffix)
- Section 30 (Model, ModelMap)

...into one unified request-handling flow.

33. Handling Login Form Submission — GET vs POST

(Note: This assumes a login form with Name/Password fields already exists in login.jsp — that specific "creating the form" step wasn't included in what you sent. Let me know if you want it added separately.)

The Problem

- Typing /login in the browser -> sends a **GET** request -> should show the login form
- Submitting the form (Name + Password + Submit) -> sends a **POST** request -> should show a **Welcome Page**

Currently, **one method** was handling both GET and POST — causing the form to just reload itself instead of navigating forward.

Discovering GET vs POST via DevTools

Right-click -> Inspect -> Network tab:

- Visiting /login in the address bar -> Request Method = **GET**
- Submitting the form -> Request Method = **POST**

Step 1: Restrict the existing method to GET only

```
@RequestMapping(value = "login", method = RequestMethod.GET)

public String gotoLoginPage() {
    return "login";
}
```

Now submitting the form (POST) to /login fails with:

Error: Method POST is not supported

This confirms the GET-only restriction worked — but we now need a **separate method for POST**.

Step 2: Create welcome.jsp

Copy login.jsp -> rename to welcome.jsp:

```
<html>

<head><title>Welcome Page</title></head>

<body>

    <div>Welcome to in28minutes</div>

</body>

</html>
```

Step 3: Create a POST-handling method

```
@RequestMapping(value = "login", method = RequestMethod.POST)

public String gotoWelcomePage(@RequestParam String name,

                                @RequestParam String password,

                                ModelMap

model) {
    model.put("name", name);
    model.put("password", password); // temporary, for testing only
    return "welcome";
}
```

Key insight: @RequestParam works for **both** query parameters (Section 30) AND **form data** — same annotation, same behavior.

Step 4: Display captured data in welcome.jsp

```
<body>

    <div>Welcome to in28minutes</div>

    <div>Your Name: ${name}</div>

    <div>Your Password: ${password}</div>

</body>
```

Result

Submit login form (Name = name, Password = dummy) -> redirected to Welcome Page showing:

Welcome to in28minutes

Your Name: name

Your Password: dummy

Key Concept: Same URL, Different HTTP Methods -> Different Behavior

```
@RequestMapping(value = "login", method = RequestMethod.GET) // show the form

@RequestMapping(value = "login", method = RequestMethod.POST) // process form submission
```

Same URL (/login) can map to **different Controller methods** depending on the **HTTP method** used.

Security Note (flagged in the transcript itself)

Displaying the password on the Welcome Page is only for testing/demonstration — this is **not secure** and should be removed. Real authentication logic comes next.

Key Takeaway Table

HTTP Method	Purpose Here	Controller Method
GET /login	Display the login form	gotoLoginPage()
POST /login	Process submitted form data, navigate	gotoWelcomePage()

HTTP Method	Purpose Here	Controller Method
-------------	--------------	-------------------

	to welcome page	
--	-----------------	--

Next: Implementing actual **authentication logic** (validating Name/Password) instead of blindly accepting any input.

34. Implementing Basic Authentication Logic

(Note: Real production apps should use **Spring Security** for authentication/authorization — covered later in the course. This is a simplified, manual version for learning purposes.)

Design Principle: Single Responsibility

Rather than putting authentication logic directly inside LoginController, create a **separate AuthenticationService** class dedicated to that responsibility.

Step 1: Create AuthenticationService

```
package
com.in28minutes.springboot.myfirstwebapp.log
in;
```

```
import
org.springframework.stereotype.Service;
```

```
@Service
```

```
public class AuthenticationService {

    public boolean authenticate(String
username, String password) {

        boolean isValidUserName =
username.equalsIgnoreCase("in28minutes");

        boolean isValidPassword =
password.equalsIgnoreCase("dummy");

        return isValidUserName &&
isValidPassword;

    }

}
```

@Service — marks this as a Spring-managed bean. Convention: use @Service (rather than

plain @Component) for classes containing **business logic**.

Hardcoded valid credentials (for this exercise): username = in28minutes, password = dummy

Step 2: Use it in LoginController via Constructor Injection

```
@Controller
```

```
public class LoginController {
```

```
    private AuthenticationService
authenticationService;
```

```
    public
LoginController(AuthenticationService
authenticationService) {
```

```
        this.authenticationService =
authenticationService;

    }
```

```
    @RequestMapping(value = "login", method
= RequestMethod.POST)
```

```
    public String
gotoWelcomePage(@RequestParam String name,
```

```
@RequestParam String password,
```

```
ModelMap
model) {
```

```
    boolean isValidUser =
authenticationService.authenticate(name,
password);
```

```
    if (isValidUser) {
        model.put("name", name);
        return "welcome";
    } else {
        model.put("errorMessage",
"Invalid Credentials! Please try again.");
        return "login";
    }

}

}
```

Errors Encountered Along the Way (and Why)

Error 1: NullPointerException on authenticationService

- Cause: Declared private AuthenticationService; but never initialized it — no new AuthenticationService() and no Spring wiring set up yet.

Error 2: APPLICATION FAILED TO START (after adding constructor)

Parameter 0 of constructor required a bean of type 'AuthenticationService'

that could not be found. Consider defining a bean of this type.

- Cause: LoginController now has a **constructor** expecting AuthenticationService -> Spring tries to **autowire it via Constructor Injection** (recall Section 11) -> but AuthenticationService isn't yet a Spring-managed bean (no @Component/@Service annotation).
- **Fix:** Add @Service to AuthenticationService -> now Spring can find and inject it.

(This is the exact same class of error explained back in Section 6 — "no bean of type X found.")

Step 3: Show error message on invalid login

In Controller:

```
model.put("errorMessage", "Invalid Credentials! Please try again.");
```

In login.jsp:

```
<pre>${errorMessage}</pre>
```

<pre> = HTML tag for preformatted text (preserves whitespace/line breaks).

Step 4: Clean up — remove password display

Since showing the password on the welcome page is insecure, it was removed from both:

- model.put("password", password) in the Controller
- The corresponding \${password} line in welcome.jsp

Testing Results

| Input | Result |
|------------------------------|---|
| in28minutes / dummy | Redirected to Welcome Page, shows Name only |
| in28minutes / wrong password | Redirected back to Login Page with "Invalid Credentials! Please try again." |

Key Takeaways

| Concept | Summary |
|--|---|
| Single Responsibility Principle | Authentication logic belongs in its own AuthenticationService, not mixed into the Controller |
| @Service | Marks a business-logic class as a Spring bean (subtype of @Component) |
| Constructor Injection | Confirmed again as the mechanism to wire dependencies — but the dependency must itself be a registered bean |
| Model for error messages | Same model.put(...) + EL (\${...}) pattern used for passing any Controller → View data, including error/validation messages |

Next (mentioned): Proper authentication/authorization using **Spring Security** — this manual approach is just a stepping stone.

35. Starting the Todo Management Application

(New feature build: Todo Management App)

Goal

Build a Todo app to manage: **Description, Target Date, Completed status (done)** — with ability to View, Add, Update, Delete Todos, plus proper page formatting.

Step 1: Create the Todo model class

```

Package:
com.in28minutes.springboot.myfirstwebapp.todo

package
com.in28minutes.springboot.myfirstwebapp.todo;

import java.time.LocalDate;

public class Todo {
    private int id;
    private String username;
    private String description;
    private LocalDate targetDate;
    private boolean done;

    public Todo(int id, String username,
String description, LocalDate targetDate,
boolean done) {
        this.id = id;
        this.username = username;
        this.description = description;
        this.targetDate = targetDate;
        this.done = done;
    }

    // Getters and Setters (generated)
    // toString() (generated)
}

```

Fields to store per Todo:

| Field | Type | Meaning |
|-------------|-----------|-------------------------|
| id | int | Unique identifier |
| username | String | Who the Todo belongs to |
| description | String | What the task is |
| targetDate | LocalDate | Deadline for the task |
| done | boolean | Whether it's completed |

Important note: Use `java.time.LocalDate` (NOT `java.util.Date`) — `LocalDate` is the modern, recommended class for storing dates in Java.

Generated via Eclipse: Constructor (using fields), Getters/Setters, `toString()` — same pattern as the earlier Course class (Section 15).

Step 2: Create `TodoService` — temporary in-memory storage

Note: Real apps store data in a database (MySQL, Oracle, etc.). This course takes a **step-by-step approach**: start with a static in-memory List, later migrate to **H2** (in-memory DB) and then **MySQL**.

```

package
com.in28minutes.springboot.myfirstwebapp.todo;

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

public class TodoService {

    private static List<Todo> todos = new
ArrayList<>();

    static {
        todos.add(new Todo(1, "in28minutes",
"Learn AWS", LocalDate.now().plusYears(1),
false));
        todos.add(new Todo(2, "in28minutes",
"Learn DevOps",
LocalDate.now().plusYears(2), false));
        todos.add(new Todo(3, "in28minutes",
"Learn Full Stack Development",
LocalDate.now().plusYears(3), false));
    }

    public List<Todo> findByUsername(String
username) {
        return todos; // simplified for now
        - returns all todos, filtering comes later
    }
}

```

```
}
```

Key points:

- todos list is **private static** — not directly accessible from outside the class
- **Static initializer block** (static { ... }) — used to populate a static field with initial values at class-loading time
- findByUsername(String username) — public method to expose Todos (currently returns **all** todos; proper filtering by username comes later)

Key Takeaway

This step is foundational/setup work:

1. **Todo** = the data model (bean)
2. **TodoService** = temporary in-memory data store + access method

Next: Build the actual page to **display** these Todos, then move toward Add/Update/Delete functionality — eventually replacing this static list with a real database (H2, then MySQL).

35. Starting the Todo Management Application

(New feature build: Todo Management App)

Goal

Build a Todo app to manage: **Description, Target Date, Completed status (done)** — with ability to View, Add, Update, Delete Todos, plus proper page formatting.

Step 1: Create the Todo model class

```
Package:
com.in28minutes.springboot.myfirstwebapp.todo

package
com.in28minutes.springboot.myfirstwebapp.todo;
```

```
import java.time.LocalDate;
```

```
public class Todo {
    private int id;
    private String username;
    private String description;
    private LocalDate targetDate;
```

```
    private boolean done;
```

```
    public Todo(int id, String username,
String description, LocalDate targetDate,
boolean done) {
        this.id = id;
        this.username = username;
        this.description = description;
        this.targetDate = targetDate;
        this.done = done;
    }
```

```
// Getters and Setters (generated)
// toString() (generated)
```

```
}
```

Fields to store per Todo:

| Field | Type | Meaning |
|-------------|-----------|-------------------------|
| id | int | Unique identifier |
| username | String | Who the Todo belongs to |
| description | String | What the task is |
| targetDate | LocalDate | Deadline for the task |
| done | boolean | Whether it's completed |

Important note: Use java.time.LocalDate (NOT java.util.Date) — LocalDate is the modern, recommended class for storing dates in Java.

Generated via Eclipse: Constructor (using fields), Getters/Setters, toString() — same pattern as the earlier Course class (Section 15).

Step 2: Create TodoService — temporary in-memory storage

Note: Real apps store data in a database (MySQL, Oracle, etc.). This course takes a **step-by-step approach**: start with a static in-memory List, later migrate to **H2** (in-memory DB) and then **MySQL**.

```
package
com.in28minutes.springboot.myfirstwebapp.todo;
```

```

import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

public class TodoService {

    private static List<Todo> todos = new
    ArrayList<>();

    static {

        todos.add(new Todo(1, "in28minutes",
        "Learn AWS", LocalDate.now().plusYears(1),
        false));

        todos.add(new Todo(2, "in28minutes",
        "Learn DevOps",
        LocalDate.now().plusYears(2), false));

        todos.add(new Todo(3, "in28minutes",
        "Learn Full Stack Development",
        LocalDate.now().plusYears(3), false));

    }

    public List<Todo> findByUsername(String
    username) {

        return todos; // simplified for now
        – returns all todos, filtering comes later

    }

}

```

Key points:

- todos list is **private static** — not directly accessible from outside the class
- **Static initializer block** (static { ... }) — used to populate a static field with initial values at class-loading time
- findByUsername(String username) — public method to expose Todos (currently returns **all** todos; proper filtering by username comes later)

Key Takeaway

This step is foundational/setup work:

1. **Todo** = the data model (bean)
2. **TodoService** = temporary in-memory data store + access method

Next: Build the actual page to **display** these Todos, then move toward Add/Update/Delete functionality — eventually replacing this static list with a real database (H2, then MySQL).

36. Building the List Todos Page

Step 1: Create TodoController

```

Package:
com.in28minutes.springboot.myfirstwebapp.todos

@Controller

public class TodoController {

    private TodoService todoService;

    public TodoController(TodoService
    todoService) {

        this.todoService = todoService;

    }

    @RequestMapping("list-todos")

    public String listAllTodos(ModelMap
    model) {

        List<Todo> todos =
        todoService.findByUsername("in28minutes");

        model.addAttribute("todos", todos);

        return "listTodos";

    }

}

```

(Note: username "in28minutes" is hardcoded for now — reason to be explained in a later step, likely once login/session integration comes in.)

Step 2: Create listTodos.jsp

Location: WEB-INF/jsp/listTodos.jsp

```

<html>

<head><title>List Todos</title></head>

<body>

    Welcome to in28minutes

    Your Todos are ${todos}

</body>

</html>

```

`{todos}` — Expression Language pulls the todos attribute from the Model (same pattern as Sections 30, 33, 34).

Errors Encountered (and fixes) — the same recurring pattern

Error 1: Consider defining a bean of type `TodoService`

- Cause: `TodoService` class existed but had no Spring annotation
- **Fix:** Add `@Service` to `TodoService` (same lesson as Sections 6 and 34)

Error 2: `NullPointerException`

- Cause: The static `List<Todo> todos` field was declared but never initialized with new `ArrayList<>()` before `.add(...)` was called on it

- **Fix:**

```
private static List<Todo> todos = new
ArrayList<>();
```

Result

`http://localhost:8080/list-todos` -> displays:

Welcome to in28minutes

Your Todos are [Todo [id=1, username=in28minutes, description=Learn AWS, ...], ...]

(Raw list printed via `toString()` — not yet formatted nicely; visual improvement comes in a later step.)

Key Takeaway — Recurring Debugging Pattern Reinforced

This step is essentially a repeat application of concepts already learned:

| Symptom | Root Cause | Fix |
|--------------------------------------|------------------------------------|--------------------------------------|
| "Consider defining a bean of type X" | Class not marked as Spring-managed | Add <code>@Component/@Service</code> |

| Symptom | Root Cause | Fix |
|---|--------------------------------------|---|
| <code>NullPointerException</code> on a collection | Field declared but never initialized | Initialize with new <code>ArrayList<>()</code> etc. |

Full Flow (Recap, applied to Todos)

GET `/list-todos`

↓

`TodoController.listAllTodos()`

↓

`todoService.findByUsername("in28minutes")` → `List<Todo>`

↓

`model.addAttribute("todos", todos)`

↓

returns "listTodos" → View Resolver → `/WEB-INF/jsp/listTodos.jsp`

↓

`{todos}` renders the list on the page

Next: Improve the page's visual formatting (proper HTML table/styling) and add more features (Add/Update/Delete Todos).

Got it — I'll tighten it up. Here's the same content, condensed:

37. Request vs Model vs Session Scope

Request Scope

DevTools Network tab shows form payload (in28minutes, dummy) sent to `/login`. This data is **gone** once the next page loads — it only exists for that one request.

Model Scope

Model attributes (Section 33) are **also request-scoped by default**. Test: name shows correctly on welcome page (from Model) but **disappears** on the next page (todos) after clicking a link — because it's a new request.

Session Scope — the fix

To persist a value across multiple requests, use `@SessionAttributes`:

```
@Controller
```

```
@SessionAttributes("name")
```

```
public class LoginController { ... }
```

```
@Controller
```

```
@SessionAttributes("name")
```

```
public class TodoController { ... }
```

Must be added to every Controller that needs access to that session value — not just where it was set.

Result: Name now persists from Login → Welcome → Todos page.

Comparison Table

| Scope | Lifetime | How to Use |
|----------------|------------------------------|---|
| Request | Single request/response only | Default (form data, Model/ModelMap) |
| Session | Across multiple requests | <code>@SessionAttributes("name")</code> on Controller |

Caution: Session data uses server memory — only store what's truly needed across pages.

38. Displaying Todos with JSTL Tags + Basic Formatting

The Problem

Expression Language (`${...}`) only displays simple values — it can't do loops/conditionals. To render a **list** of Todos as a table, we need **JSTL** (JSP Standard Tag Library).

Step 1: Add JSTL dependencies to pom.xml

```
<dependency>

<groupId>jakarta.servlet.jsp.jstl</groupId>

<artifactId>jakarta.servlet.jsp.jstl-api</artifactId>

<version>3.0.0</version>

</dependency>
```

```
<dependency>

<groupId>org.glassfish.web</groupId>

<artifactId>jakarta.servlet.jsp.jstl</artifactId>

<version>3.0.1</version>

</dependency>
```

(pom.xml change → requires manual Stop + Start of the server.)

Step 2: Import the tag library in the JSP

```
<%@ taglib prefix="c"
uri="http://java.sun.com/jsp/jstl/core" %>
```

`c` = prefix used to reference JSTL core tags (forEach, if, catch, etc.) in the page.

Step 3: Loop and render Todos in a table (listTodos.jsp)

```
<table>

<thead>

<tr>

<th>Todo Id</th>

<th>Description</th>

<th>Target Date</th>

<th>Is Done?</th>

</tr>

</thead>

<tbody>

<c:forEach var="todo" items="${todos}">

<tr>

<td>${todo.id}</td>

<td>${todo.description}</td>

<td>${todo.targetDate}</td>

<td>${todo.done}</td>

</tr>

</c:forEach>

</tbody>

</table>
```

c:forEach = JSTL equivalent of an enhanced for-loop — iterates `items="${todos}"`, exposing each element as `var="todo"`.

Step 4: Light formatting cleanup

```
listTodos.jsp:
Welcome
<hr>
<h1>Your Todos</h1>

welcome.jsp:
Welcome ${name}
<hr>
```

Result

Full flow now works: Login → Welcome (Welcome in 28 minutes) → Manage Todos → Todos table displayed properly with headers and rows.

Key Takeaway

| Tool | Purpose |
|-------------------------------|--|
| Expression Language \${...} | Display simple values |
| JSTL (c:forEach, etc.) | Loops/conditionals for dynamic content in JSPs |

39. Adding Bootstrap via WebJars

Why WebJars

Instead of manually downloading CSS/JS files, **WebJars** lets Maven download front-end libraries (like Maven does for any dependency) — auto-managed via pom.xml.

Add to pom.xml

```
<dependency>
    <groupId>org.webjars</groupId>
    <artifactId>bootstrap</artifactId>
    <version>5.1.3</version>
</dependency>
<dependency>
    <groupId>org.webjars</groupId>
    <artifactId>jquery</artifactId>
    <version>3.6.0</version>
</dependency>
```

Note: Unlike Spring dependencies (version managed by spring-boot-starter-parent),

WebJars need an **explicit version**. (*pom.xml change → stop/start server required.*)

Files land under

META-INF/resources/webjars/bootstrap/5.1.3/css/bootstrap.min.css

META-INF/resources/webjars/bootstrap/5.1.3/js/bootstrap.min.js

META-INF/resources/webjars/jquery/3.6.0/jquery.min.js

Reference them in JSP

CSS (in <head>):

```
<link
href="webjars/bootstrap/5.1.3/css/bootstrap.min.css" rel="stylesheet">
```

JS (just before </body>):

```
<script
src="webjars/jquery/3.6.0/jquery.min.js"></script>

<script
src="webjars/bootstrap/5.1.3/js/bootstrap.min.js"></script>
```

(No need to prefix with /META-INF/resources — Spring Boot serves from there automatically.)

Verification

DevTools → Network tab shows separate requests for list-todos, bootstrap.min.css, bootstrap.min.js, jquery.min.js — browser auto-fetches linked assets referenced in the HTML response.

Note: Bootstrap 5.1.3 / jQuery 3.6.0 were current at recording time — check for newer versions if applying this today.

Next: Using Bootstrap classes to actually style the page.

40. Applying Bootstrap Classes to Pages

listTodos.jsp

Wrap all body content in Bootstrap's recommended container:

```
html
<div class="container">
  <h1>Your Todos</h1>
  <table class="table">
    ...
  </table>
</div>
```

- **class="container"** — Bootstrap's standard wrapper class; centers content with proper margins (without it, content sits flush left)
- **class="table"** — Bootstrap's table styling class; instantly formats the table (borders, spacing, etc.)

login.jsp

```
html
<div class="container">
  <h1>Login</h1>
  <!-- error message display -->
  <!-- login form -->
</div>
```

welcome.jsp

```
html
<div class="container">
  <h1>Welcome ${name}</h1>
  Manage your todos
</div>
```

Result

All three pages (Login, Welcome, List Todos) now have consistent, centered, Bootstrap-styled layout — a noticeable visual improvement over plain unstyled HTML.

Key Takeaway

Two simple Bootstrap classes did most of the work:

Class	Effect
container	Centers page content, adds proper margins

Class	Effect
table	Applies clean table styling (borders, spacing)

Next: Move on to adding full CRUD functionality — Add, Delete, Update Todos.

41. Creating the Add Todo Page (GET) and Form

Step 1: Add a button on listTodos.jsp

```
html
<a href="add-todo" class="btn btn-success">Add Todo</a>
```

- btn = base Bootstrap button class
- btn-success = green styling, signals a positive/primary action

Step 2: Create GET handler in TodoController

```
java
@RequestMapping(value = "add-todo", method = RequestMethod.GET)
public String showNewTodoPage() {
    return "todo"; // generic view name, reused for add & later edit
}
```

Step 3: Create todo.jsp

```
html
<html>
<head><title>Add Todo Page</title></head>
<body>
  <h2>Enter Todo Details</h2>
  <form method="post">
    <input type="text"
name="description">
    <input type="submit" class="btn btn-success">
  </form>
</body>
</html>
```

Debugging Journey

Problem 1: Form submission just reloads the same page

- Cause: The add-todo mapping had **no method restriction** — it handled GET and POST identically, so both just returned the same "todo" view.
- **Fix:** Restrict the GET handler explicitly:

```
java
```

```
@RequestMapping(value = "add-todo", method = RequestMethod.GET)
```

- Result: submitting now correctly throws **"Request method 'POST' is not supported"** — confirms GET-only restriction worked (same debugging pattern as Section 33).

Problem 2: Need a separate POST handler

```
java
```

```
@RequestMapping(value = "add-todo", method = RequestMethod.POST)
```

```
public String addNewTodo() {
    return "listTodos"; // temporary – no
    processing logic yet
}
```

By default, a form's POST submits to the **same URL** it was served from (add-todo).

Problem 3: Redirecting to listTodos view directly shows a blank/empty page

- Cause: Returning the **view name** "listTodos" directly re-renders the JSP **without repopulating the model** — so `${todos}` has no data.
- **Attempted fix (rejected):** Manually re-fetch the todo list and add to ModelMap again — but this **duplicates logic** already in the other controller method.

Problem 4 (after using redirect:): "Page not found"

```
java
```

```
return "redirect:listTodos"; // WRONG — uses
JSP name, not URL
```

- Cause: Used the **view name** (listTodos) instead of the actual **URL** (list-todos).
- **Fix:**

```
java
```

```
return "redirect:list-todos"; // correct —
matches the actual @RequestMapping URL
```

The redirect: Prefix — Key New Concept

```
java
```

```
return "redirect:list-todos";
```

Instead of directly rendering a view, **redirect:** tells Spring MVC to send an **HTTP redirect** back to the browser, causing it to make a **brand-new GET request** to /list-todos. This new request goes through the normal `listAllTodos()` method — which correctly fetches and populates the todos in the Model.

Why this matters: Avoids duplicating "fetch todos + populate model" logic across multiple controller methods — the existing GET /list-todos handler is reused.

Final Working Flow

listTodos page → click "Add Todo" (GET /add-todo) → todo.jsp form

→ Submit (POST /add-todo) → `addNewTodo()` → `redirect:list-todos`

→ browser issues new GET /list-todos → `listAllTodos()` → todos displayed

Key Takeaway

Concept	Summary
Same URL, different HTTP methods	<code>RequestMethod.GET</code> shows the form, <code>RequestMethod.POST</code> processes it
<code>redirect:</code> prefix	Triggers a new browser request to a URL, instead of directly rendering a view — avoids duplicating data-population logic
Must use the URL , not the view name , after <code>redirect:</code>	<code>redirect:list-todos</code> (URL) ≠ <code>redirect:listTodos</code> (JSP/view name)

42. Adding a New Todo (Business Logic)

Add logic to TodoService

```

java

private static int todosCount = 3; // 3
todos already exist

public void addTodo(String name, String
description, LocalDate targetDate, boolean
done) {

    Todo todo = new Todo(++todosCount, name,
description, targetDate, done);

    todos.add(todo);
}

```

++todosCount — pre-increment ensures each new Todo gets a unique, incrementing id.

Wire it up in TodoController

```

java

@RequestMapping(value = "add-todo", method =
RequestMethod.POST)

public String addNewTodo(ModelMap model,
@RequestParam String description) {

    String username = (String)
model.get("name");

    todoService.addTodo(username,
description, LocalDate.now().plusYears(1),
false);

    return "redirect:list-todos";
}

```

- username retrieved from the Model (set earlier via @SessionAttributes("name") — Section 37)
- targetDate hardcoded to 1 year from now for now (a proper date-picker field comes later)
- done hardcoded to false (new Todos always start incomplete)
- Uses redirect:list-todos (established in Section 41) to reuse the existing GET handler that populates the Model correctly

Testing

Login → Manage Todos → Add Todo → "Learn Google Cloud" → Submit → new Todo appears in the list with correct target date.

Known Limitations (flagged for later)

1. **Data is not persistent** — todos is a static in-memory list, so **restarting the server wipes all added Todos**. Will be fixed later using a real database (MySQL).
2. **No validation** — submitting an **empty description** is currently accepted without error.

Key Takeaway

Adding new data follows the layered pattern used throughout the course:

Form submission (POST) → Controller (@RequestParam) → Service (business logic) → in-memory list update → redirect to list-todos

43. Server-Side Validation (Part 1) — Command Bean / Form Backing Object

Why not rely on front-end validation alone?

Adding required="required" on an HTML input (front-end validation) works, but:

Front-end/JavaScript validations can easily be bypassed by hackers. Server-side validation in Java is the required best practice.

The 4 Steps to Server-Side Validation (Spring Boot)

1. Add spring-boot-starter-validation to pom.xml
2. Use a **Command Bean / Form Backing Object** (enables **2-way binding**)
3. Add validation annotations to the bean (Todo.java)
4. Display validation errors in the view (JSP)

(This step covers #1 and #2 only — #3 and #4 come next.)

Step 1: Add the validation starter

```

xml

<dependency>

<groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-
validation</artifactId>

```

```
</dependency>
```

(pom.xml change → Stop + Start server required.)

Step 2: Command Bean (Form Backing Object)

The Problem: Using @RequestParam per field doesn't scale — 10 form fields would need 10 separate @RequestParam.

The Solution: Bind the entire form directly to the Todo bean instead.

Controller — bind to Todo directly:

```
java

@RequestMapping(value = "add-todo", method =
RequestMethod.POST)

public String addNewTodo(ModelMap model,
Todo todo) {

    todoService.addTodo(todo.getDescription(),
...);

    return "redirect:list-todos";
}
```

Todo todo as a parameter (no @RequestParam) — Spring MVC automatically binds matching form field names to the bean's properties.

Using Spring's Form Tag Library in JSP

```
jsp

<%@ taglib prefix="form"
uri="http://www.springframework.org/tags/for
m" %>
```

Updated form using form:form and form:input:

```
jsp

<form:form method="post"
modelAttribute="todo">

    <form:input path="description"
required="required"/>

    <input type="submit" class="btn btn-
success"/>

</form:form>
```

- **modelAttribute="todo"** — ties the form to a model attribute named "todo" (must match exactly what's put into the Model)

- **form:input path="description"** — binds this input directly to the description field of the Todo bean

Error Encountered: Missing model attribute

Neither BindingResult nor plain target object for bean 'todo'

available as request attribute

Cause: The GET handler (showNewTodoPage) never added a "todo" object to the Model — but the JSP's form:form expects one.

Fix — populate a blank/default Todo in the GET handler:

```
java

@RequestMapping(value = "add-todo", method =
RequestMethod.GET)

public String showNewTodoPage(ModelMap
model) {

    String username = (String)
model.get("name");

    Todo todo = new Todo(0, username, "",
LocalDate.now().plusYears(1), false);

    model.put("todo", todo);

    return "todo";
}
```

Second Error: Validation failure on hidden fields

Validation failed for object='todo'. Error count: 2

- id: null

- done: null

Cause: The form only had an input for description — id and done weren't submitted, so they came back as null when the form posted.

Fix — add hidden inputs for id and done:

```
jsp

<form:input path="id" type="hidden"/>

<form:input path="done" type="hidden"/>
```

Why hidden, not visible text fields: These values shouldn't be user-editable — hidden

inputs still submit the value but don't render visibly/editably in the form.

Result

Form now only visibly shows the **Description** field; id and done are passed along invisibly. Submitting successfully creates a new Todo.

Key Concepts Introduced

Concept	Meaning
Command Bean / Form Backing Object	A Java bean (e.g., Todo) used to directly bind an entire form's fields, instead of individual @RequestParams
2-way binding	The same bean is used both to populate the form (GET, pre-filled values) and to receive submitted data (POST)
form:form / form:input	Spring's JSP tag library for binding form fields directly to bean properties via path
Hidden fields	Used to pass required-but-not-user-editable values (like id, done) through the form

44. Server-Side Validation (Part 2) — 2-Way Binding, Validation Annotations & Displaying Errors

Understanding 2-Way Binding (recap/demo)

- **Bean → Form:** Setting a default value in the Todo object (e.g., "Default Desc") in the GET handler automatically shows up in the form field when the page loads.
- **Form → Bean:** Typing "Learning DevOps" and submitting binds that value back into the Todo object's description field on the POST request.

2-way binding = the same bean drives both directions: populating the form AND receiving submitted data. This is why using a Command Bean scales well — adding a new field (e.g., targetDate) later just works without extra wiring.

Step 3: Add Validation Annotations to Todo.java

```
java
import jakarta.validation.constraints.Size;

public class Todo {
    private int id;
    private String username;

    @Size(min = 10, message = "Enter at least 10 characters")
    private String description;

    private LocalDate targetDate;
    private boolean done;
    // ...
}
```

Other available validation annotations (from jakarta.validation.constraints, brought in by spring-boot-starter-validation):

Annotation	Checks
@Size(min=, max=)	String/collection length bounds
@NotNull	Value must not be null
@NotBlank	String must not be null/empty/whitespace
@Future	Date must be in the future
@FutureOrPresent	Date must be today or later
@Past	Date must be in the past

Step 4: Trigger validation with @Valid

```
java
@RequestMapping(value = "add-todo", method = RequestMethod.POST)

public String addNewTodo(ModelMap model,
    @Valid Todo todo, BindingResult result) {
    if (result.hasErrors()) {
```

```

        return "todo"; // validation failed
    }
    // go back to the form
    }

    todoService.addToDo(...);

    return "redirect:list-todos";
}

```

Key pieces:

- **@Valid** (from jakarta.validation.Valid) — tells Spring to actually run the validation annotations on Todo before proceeding
- **BindingResult result** — captures validation errors; **must be declared immediately after** the @Valid object in the method signature
- **result.hasErrors()** — check before proceeding; if true, return back to the form view instead of saving/redirecting

Without @Valid: annotations like @Size are silently ignored — validation never triggers.

Displaying Validation Errors in the JSP

jsp

```

<form:input path="description"
required="required"/>

<form:errors path="description"
cssClass="text-warning"/>

```

- **form:errors** (not form:error) — Spring's tag for displaying validation error messages tied to a specific field via path
- **cssClass** — Spring tag equivalent of HTML's class attribute (can't use class= directly on Spring tags). text-warning is a Bootstrap class that styles text as a warning (typically yellow/orange).

Testing Results

Input	Result
"Learn AWS" (9 chars) — without @Valid	No validation triggered, saved anyway (bug)
"Learn" (5 chars) — with @Valid + BindingResult	Validation error shown: "Enter at least 10 characters"

Input	Result
"Learn Google Cloud" (19 chars)	Passes validation, saved successfully

Complete Validation Flow

```

POST /add-todo
↓
@Valid Todo todo → runs @Size and other
                    annotations on Todo fields
↓
BindingResult result → captures any
                    validation failures
↓
if (result.hasErrors())
    → return "todo" (redisplay form with
                    errors via <form:errors>)
else
    → save + redirect:list-todos

```

Key Takeaways

Concept	Summary
2-way binding	Command Bean handles both displaying form data AND receiving submitted data
@Size, @NotNull, etc.	Declarative validation rules placed directly on bean fields
@Valid	Activates validation checking on that parameter
BindingResult	Must immediately follow the @Valid parameter; holds validation outcome
<form:errors path="..." cssClass="...">	Displays field-specific validation messages in the JSP

45. Cleanup Before Spring Security

(Prep step — Spring Security will handle login/logout, so manual login code needs to be removed first.)

Changes to LoginController → renamed WelcomeController

Before: Handled /login (GET/POST) with manual authentication logic (Sections 33-34).

After:

```
java
@Controller
public class WelcomeController {

    @RequestMapping("/")
    public String gotoWelcomePage(ModelMap
model) {
        model.put("name", "in28minutes"); //
        hardcoded for now, fixed later
        return "welcome";
    }
}
```

Cleanup checklist performed

1. **Changed URL mapping** from login to / — root URL now goes straight to the welcome page
2. **Hardcoded username** ("in28minutes") temporarily — real login will come from Spring Security later
3. **Removed all authentication logic** (the authenticationService.authenticate(...) calls)
4. **Deleted AuthenticationService.java** entirely — no longer needed
5. **Deleted login.jsp** — no longer needed
6. **Renamed method:** gotoLoginPage() → gotoWelcomePage()
7. **Renamed class:** LoginController → WelcomeController

Key Takeaway

When refactoring, **rename everything consistently** — method names, class names, and file names should all reflect the new purpose. Leftover names like gotoLoginPage in a class that no longer does login would confuse future readers of the code.

Verification

http://localhost:8080/ → shows "Welcome in28minutes" → "Manage your todos" still works → Add/Update Todo flows still work.

46. Introducing Spring Security

Adding Spring Security

```
xml
<dependency>

<groupId>org.springframework.boot</groupId>

    <artifactId>spring-boot-starter-
security</artifactId>
</dependency>
```

(pom.xml change → Stop + Start server required.)

What Happens Immediately (Auto-Configuration in action)

On restart, console logs show:

Using generated security password: <some-random-password>

This generated password is for development use only.

Effect: Every URL in the app is now **automatically protected**. Trying to visit /add-todo, /list-todos, or even / redirects to a **built-in login page**.

Default Login

- **Username:** user
- **Password:** the auto-generated one shown in the console logs (different every restart)

Logging in with these credentials grants access to the app.

Key Observation

The **Spring Security user** (user / generated password) is completely separate from the **application's own concept of a user** (e.g., in28minutes). Right now, Spring Security is just protecting the URLs — it isn't yet integrated with app-specific user data.

Problem with the Default Setup

The auto-generated password **changes every time the server restarts** — not usable for real development/testing.

Key Takeaway

Adding just **one dependency** (spring-boot-starter-security) instantly secures **all** endpoints via Spring Boot's auto-configuration (recall Section 18) — zero manual configuration needed to get basic protection.

Next: Configure **custom users and passwords** instead of relying on the auto-generated one.

47. Configuring Custom Spring Security Users

Step 1: Create a Security Configuration class

```
Package:
com.in28minutes.springboot.myfirstwebapp.security

java

@Configuration

public class SpringSecurityConfiguration {

    @Bean
    public InMemoryUserDetailsManager
configureUserDetails() {

        UserDetails userDetails =
User.withDefaultPasswordEncoder()

            .username("in28minutes")
            .password("dummy")
            .roles("USER", "ADMIN")
            .build();

        return new
InMemoryUserDetailsManager(userDetails);
    }
}
```

Key points:

- **@Configuration** — marks the class as a source of Spring Bean definitions (Beans are created directly inside methods, unlike @Component)
- **InMemoryUserDetailsManager** — stores user credentials in memory (real apps typically use LDAP or a database)

- **UserDetails** is an interface — built via the User builder class
- **.roles("USER", "ADMIN")** — assigns roles used for authorization (controlling what actions a user can perform), separate from authentication (login credentials)
- **withDefaultPasswordEncoder()** is **deprecated** — flagged for replacement

Testing

- Spring Security auto-provides a **logout URL** (no extra code needed)
- Login with in28minutes / dummy → works
- This is now the app's **own** configured user (replacing the auto-generated user/random-password from Section 46)

Step 2: Replace deprecated encoder with BCryptPasswordEncoder

Add a PasswordEncoder bean:

```
java

@Bean

public PasswordEncoder passwordEncoder() {

    return new BCryptPasswordEncoder();

}
```

BCryptPasswordEncoder — implementation of PasswordEncoder using the **BCrypt** strong hashing algorithm (industry standard for password hashing).

Update user configuration to use it:

```
java

@Bean

public InMemoryUserDetailsManager
configureUserDetails(PasswordEncoder
passwordEncoder) {

    UserDetails userDetails = User.builder()

        .username("in28minutes")

        .password(passwordEncoder.encode("dummy"))
        .roles("USER", "ADMIN")
```

```

        .build();

    return new
    InMemoryUserDetailsManager(userDetails);
}

```

Error Encountered Along the Way

"Bad credentials" after adding BCryptPasswordEncoder but before updating user config:

- **Cause:** Spring Security now expects passwords to be **BCrypt-encoded**, but the stored password was still using the old default encoder — mismatch.
- **Fix:** Encode the password explicitly using the same passwordEncoder when building UserDetails (as shown above).

(Transcript mentions a Lambda-based approach for the encoder function as an alternative syntax — functionally equivalent to calling passwordEncoder.encode(input).)

Final Verification

- Login with in28minutes / dummy → works
- Homepage, Todos list, Add/Update/Delete, Logout — all functioning
- After logout, visiting a protected URL (e.g., /list-todos) directly → redirected to login page (confirms security is enforced)

Key Takeaway

Concept	Summary
@Configuration + @Bean	Manually define Spring Beans (here: security-related beans)
InMemoryUserDetailsManager	Simple in-memory store for usernames/passwords/roles (dev/demo use)
User.builder()	Constructs UserDetails objects

Concept	Summary
.roles(...)	Defines authorization roles (separate from authentication)
BCryptPasswordEncoder	Recommended way to securely hash passwords — never store/compare plain text
Password must be encoded consistently	Same encoder must be used both when storing and when verifying passwords
Next: Likely deeper Spring Security topics — role-based authorization, securing specific URLs/methods, or integrating security with the app's actual user data (recall the earlier hardcoded "in28minutes" in WelcomeController).	

48. Removing Hardcoded Username — Using Spring Security Context

The Problem

"in28minutes" was hardcoded in multiple places (WelcomeController, TodoController), even though Spring Security now manages actual authenticated users.

Getting the Logged-in User from Spring Security

```

java
private String getLoggedInUsername() {
    Authentication authentication =
    SecurityContextHolder.getContext().getAuthen
    tication();

    return authentication.getName();
}

```

- **SecurityContextHolder.getContext().getAuthentication()** — retrieves the currently authenticated user's details from Spring Security
- **authentication.getName()** — returns the actual username of whoever is logged in

Update WelcomeController

```
java
@RequestMapping("/")
public String gotoWelcomePage(ModelMap
model) {
    model.put("name",
getLoggedInUsername()); // replaces
hardcoded "in28minutes"

    return "welcome";
}
```

Update TodoController

Same getLoggedInUsername() method extracted here too (via **Eclipse Refactor → Extract Method**, which found and updated multiple usages at once).

```
java
@RequestMapping("list-todos")
public String listAllTodos(ModelMap model) {
    String username = getLoggedInUsername();

    model.addAttribute("todos",
todoService.findByUsername(username));

    return "listTodos";
}
```

Bonus fix: Actually filter Todos by username in TodoService

Previously findByUsername ignored the parameter and returned **all** todos. Now properly filtered:

```
java
public List<Todo> findByUsername(String
username) {
    Predicate<Todo> predicate = todo ->
todo.getUsername().equalsIgnoreCase(username
);

    return
todos.stream().filter(predicate).collect(Col
lectors.toList());
}
```

- **Predicate<Todo>** — a lambda condition: returns true if a Todo's username matches
- **.stream().filter(predicate).collect(Collectors.toList())** — standard Java

Streams pattern: convert list → stream → filter by condition → collect back into a list

- Uses **.equalsIgnoreCase()** since usernames are Strings (can't use **==**)

Why Get Username from Spring Security Directly (not from Model/Session)?

Bug scenario: If a user logs in and navigates **directly** to /list-todos (bypassing WelcomeController's / route), the @SessionAttributes("name") value would **never get set** — because that code path never executed.

Fix: Always fetch the username **directly from SecurityContextHolder** in each Controller that needs it, rather than relying on it having been stored in session by another controller.

Key Takeaway

Before	After
Hardcoded "in28minutes" string	SecurityContextHolder.getContext().getAuthentication().getName()
findByUsername() returned all todos (no filtering)	Properly filters using Java Streams + Predicate
Relied on session value being set by another Controller	Each Controller independently fetches the authenticated user — more robust

This completes the integration between **Spring Security's authentication** and the **application's actual user-specific data** — todos are now correctly scoped to whoever is logged in.

49. Adding Multiple Users to Spring Security

Refactoring for Reusability

Used Eclipse's **Refactor** → **Extract Method** to turn the single-user creation code into a reusable method:

```
java

private UserDetails createNewUser(String
username, String password) {

    return User.builder()

        .username(username)

        .password(passwordEncoder().encode(password)
        )

        .roles("USER", "ADMIN")

        .build();

}
```

Creating Multiple Users

```
java

@Bean

public InMemoryUserDetailsManager
configureUserDetails() {

    UserDetails userDetails1 =
createNewUser("in28minutes", "dummy");

    UserDetails userDetails2 =
createNewUser("ranga", "dummydummy");

    return new
InMemoryUserDetailsManager(userDetails1,
userDetails2);

}
```

Key detail: InMemoryUserDetailsManager's constructor accepts **varargs** (UserDetails...) — so any number of users can be passed in.

Testing Multi-User Behavior

1. Login as ranga / dummydummy → Todos list starts **empty** (correctly scoped, since findByUsername filters by the logged-in user — Section 48)
2. Add Todos as ranga ("Learn AWS Ranga", "Get AWS Certified Now Ranga") → they appear correctly for ranga
3. Open a **separate browser** → login as in28minutes / dummy → sees their own

original Todos, unaffected by ranga's additions

Result

Confirms the app now properly supports **multiple independent users**, each seeing only their own Todos — validating the earlier work in Sections 47-48 (custom Spring Security users + username-based filtering).

Key Takeaway

Adding a new user is now trivial: just call createNewUser(username, password) again and pass it into InMemoryUserDetailsManager. Combined with the username-based filtering in TodoService, the app correctly isolates data per user.

50. Introducing a Database — H2 Setup (Part 1)

(New phase: replacing the static in-memory List<Todo> with a real database, starting with H2, later MySQL.)

Add Dependencies

<dependency>

<groupId>org.springframework.boot</groupId>

<artifactId>spring-boot-starter-data-jpa</artifactId>

</dependency>

<dependency>

<groupId>com.h2database</groupId>

<artifactId>h2</artifactId>

<scope>runtime</scope>

</dependency>

- **spring-boot-starter-data-jpa** — Spring Boot's starter for connecting to databases via **JPA** (Java Persistence API)
- **h2** — an **in-memory database**, useful for development/testing without needing a real DB server
- **scope=runtime** — H2 is only needed while the app is running, not needed at compile time

(pom.xml change → Stop + Start server required. Troubleshooting tip mentioned: right-click project → Maven → Update Project → check "Force Updates of Snapshots and Releases" if dependencies don't resolve properly.)

What Happens on Startup

Console logs show:

H2 console available at '/h2-console'

Database available at 'jdbc:h2:mem:<random-generated-name>'

Problem: The database URL is **randomly generated** each restart — not usable for consistent configuration.

Fixing the URL — application.properties

spring.datasource.url=jdbc:h2:mem:testdb

Now the database is always accessible at a **fixed, predictable** URL: jdbc:h2:mem:testdb.

Accessing H2 Console

1. Visit: http://localhost:8080/h2-console
2. (Protected by Spring Security — must log in first, e.g., in28minutes/dummy)
3. On the H2 login screen: JDBC URL = jdbc:h2:mem:testdb, Username = sa, Password = (empty)
4. Click **Connect**

Error Encountered

Connecting to H2 console fails — caused by a **Spring Security configuration conflict** (H2 console runs inside a frame/iframe, which Spring Security blocks by default, and its URL also needs to be excluded from the normal authentication flow).

Key Takeaway

Concept	Purpose
H2	Lightweight in-memory database — good for development, resets on restart

Concept	Purpose
JPA (spring-boot-starter-data-jpa)	Spring's abstraction for talking to relational databases
spring.datasource.url	Fixes the database connection URL so it's consistent across restarts
H2 Console (/h2-console)	Web-based UI to inspect/query the H2 database directly
Next: Fix the Spring Security configuration so the H2 console can be accessed properly.	

51. Fixing Spring Security to Allow H2 Console Access

The Problem

By default, Spring Security provides two behaviors:

1. **All URLs are protected** (require authentication)
2. **Unauthorized requests show a login form**

But H2 Console needs two additional adjustments:

1. **CSRF (Cross-Site Request Forgery) protection must be disabled** for it to work
2. **Frame usage must be allowed** — H2's console UI uses HTML frames, which Spring Security blocks by default

Configuring a Custom SecurityFilterChain

@Configuration

```
public class SpringSecurityConfiguration {

    @Bean
    public SecurityFilterChain
filterChain(HttpSecurity http) throws
Exception {

        http.authorizeHttpRequests(auth ->
auth.anyRequest().authenticated());

        http.formLogin(withDefaults());
    }
}
```

```

        http.csrf(csrf -> csrf.disable());

        http.headers(headers ->
headers.frameOptions(frame ->
frame.disable()));

        return http.build();
    }
}

```

Import needed (static import):

```

import static
org.springframework.security.config.Customiz
er.withDefaults;

```

Breaking Down Each Line

Line	Purpose
authorizeHttpRequests(auth -> auth.anyRequest().authenticated())	Re-establishes: all requests require authentication (the default behavior, now explicitly configured)
formLogin(withDefaults())	Re-establishes: show the default login form for unauthenticated users
csrf(csrf -> csrf.disable())	Disables CSRF protection — required for H2 console to function
headers(headers -> headers.frameOptions(frame -> frame.disable()))	Allows frames — required since H2 console UI uses HTML frames

Key concept — HttpSecurity: Configures web-based security rules; applies to all requests by default. SecurityFilterChain is the bean that

intercepts every incoming HTTP request and applies these rules.

Important: Since we're now **defining our own** SecurityFilterChain bean, we must **explicitly re-configure** the default behaviors (authentication + form login) that Spring Boot was providing automatically before — overriding the default replaces it entirely, so nothing is inherited unless restated.

Fixing the H2 Console Connection

After the security fix, connecting still failed initially because the JDBC URL field showed a different default value (test) instead of the configured one.

Fix: Manually enter the correct URL matching application.properties:

```
jdbc:h2:mem:testdb
```

Username: sa, Password: (*empty*) → **Connect** → success.

Key Takeaway

Concept	Summary
SecurityFilterChain	The bean that intercepts and processes every HTTP request for security rules
Overriding it means redefining everything	Default behaviors (auth required, login form) must be explicitly restated
CSRF disabled + Frames allowed	Specifically needed to make the H2 Console functional
Always match the exact configured DB URL	application.properties value must match what's typed into the H2 console login screen

Next: With H2 Console access working, the course moves on to actually **creating tables and connecting via JPA** to persist Todos in a real database.

52. Overview — JDBC vs Spring JDBC vs JPA vs Spring Data JPA

(High-level conceptual step — no code changes yet.)

The Evolution of Database Access in Java

1. JDBC (Java Database Connectivity)

- Requires writing **raw SQL queries** (SELECT, INSERT, UPDATE, DELETE) AND a lot of **boilerplate Java code** to execute them
- Historically painful — "thousands of lines of code" for basic operations

2. Spring JDBC

- A Spring module — still requires **writing SQL queries manually**
- BUT reduces the amount of **boilerplate Java code** needed to execute them
- Example: `jdbcTemplate.update(query, id)` — much less code than raw JDBC, but the query itself is still hand-written

3. JPA (Java Persistence API) / Hibernate

- **No manual SQL queries needed**
- Instead: **map Java classes (Entities) to database tables** using annotations
- JPA/Hibernate **automatically generates the SQL** based on this mapping
- Example: finding a record by ID becomes pure Java code — no query written at all

4. Spring Data JPA

- Builds on top of JPA to make it **even simpler**
- Just define an **interface extending JpaRepository**
- Get automatic CRUD operations (Create, Retrieve, Update, Delete) — **no implementation code needed at all**

Comparison Table

Approach	Write SQL manually ?	Java boilerplate	Complexity
JDBC	Yes	High	Highest
Spring JDBC	Yes	Reduced	High
JPA / Hibernate	No (auto-generated)	Low	Medium
Spring Data JPA	No	Minimal (just an interface)	Lowest

Key Takeaway

The overall trend: move **away from manually writing SQL and boilerplate Java code**, toward **declarative mapping** (JPA) and eventually **near-zero-code interfaces** (Spring Data JPA).

This course will use **Spring Data JPA** — matching the earlier addition of `spring-boot-starter-data-jpa` (Section 50).

Next: Actually implementing JPA entity mapping and Spring Data JPA repositories for the `Todo` class.

53. Mapping Todo to a Database Table with JPA @Entity

Step 1: Mark Todo as a JPA Entity

```
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.GeneratedValue;
```

```
@Entity
public class Todo {

    @Id
    @GeneratedValue
    private int id;
```

```
    private String username;
    private String description;
```



```

private LocalDate targetDate;
private boolean done;

// ... constructors, getters, setters
}

```

Key annotations:

Annotation	Purpose
@Entity	Maps this Java class to a database table
@Id	Marks the field as the primary key
@GeneratedValue	Auto-generates the ID value (e.g., via a sequence)

Auto-Table Creation (Spring Boot Magic)

As soon as Spring Boot Auto Configuration detects **H2 in the classpath** + an @Entity class, it **automatically creates a matching table** — no manual CREATE TABLE needed.

Verified by refreshing H2 Console — a TODO table appears automatically with columns: ID, DESCRIPTION, DONE, TARGET_DATE, USERNAME.

Customizing Table/Column Names (optional, demonstrated then reverted)

```

@Entity(name = "TodoABC") // custom table
name (demo only, later removed)

@Column(name = "name") // custom column
name (demo only, later removed)

private String username;

```

By default, JPA uses the **class name** for the table and **field name** for columns.

Naming convention note: targetDate (camelCase) automatically becomes TARGET_DATE in the database — JPA converts camelCase field names to **snake_case with underscores** for column names.

Step 2: Pre-populate Data with data.sql

Create: src/main/resources/data.sql

```

INSERT INTO todo (id, username, description,
target_date, done)

```

```

VALUES (10001, 'in28minutes', 'Get AWS
Certified', CURRENT_DATE, false);

```

```

INSERT INTO todo (id, username, description,
target_date, done)

```

```

VALUES (10002, 'in28minutes', 'Get Azure
Certified', CURRENT_DATE, false);

```

- Explicitly listing column names (id, username, description, target_date, done) avoids ambiguity/ordering issues
- **Must use single quotes** for string values (not double quotes) — using double quotes caused a Column not found syntax error
- IDs must be **unique** (avoid clashing with the auto-generated sequence)

Error Encountered: Table TODO not found

Cause: By default, data.sql runs **before** JPA creates the tables from @Entity classes — so the INSERT fails because the table doesn't exist yet.

Fix — application.properties:

```
spring.jpa.defer-datasource-initialization=true
```

This **defers** running data.sql until **after** JPA has created the schema/tables.

Verification

After restart: H2 Console → TODO table now contains the pre-populated data.

Important Reminder

H2 is in-memory — restarting the application **wipes all data** again (same limitation flagged for the static list back in Section 42, just now at the database level instead).

Key Takeaway

Concept	Summary
@Entity	Maps a Java class to a DB table
@Id + @GeneratedValue	Defines primary key with auto-generated values

Concept	Summary	Method	Purpose
Auto table creation	Spring Boot + H2 + @Entity = automatic schema generation	getById(id)	Retrieve by primary key
data.sql	Pre-populates data at startup	.save(entity)	Insert or update
spring.jpa.defer-datasource-initialization=true	Ensures tables exist before data.sql runs	.delete(entity)	Delete by object
camelCase → snake_case	Default JPA column naming convention	.deleteById(id)	Delete by primary key
...and many more			Full CRUD without writing implementation code
Step 2: Create a parallel Controller to experiment safely - Copied TodoController → TodoControllerJpa - Commented out @Controller on the original TodoController — this makes Spring ignore it entirely (no duplicate @RequestMapping conflicts), keeping it as a backup/reference while TodoControllerJpa is developed			
Step 3: Wire TodoRepository into the new Controller (Constructor Injection) <pre>@Controller public class TodoControllerJpa { private TodoRepository todoRepository; public TodoControllerJpa(TodoRepository todoRepository) { this.todoRepository = todoRepository; } // ... }</pre>			
Step 4: Add a custom query method — findByUsername JpaRepository only provides ID-based lookups out of the box. To search by a different field (username), declare a method matching Spring Data JPA's naming convention — no implementation needed: <pre>public interface TodoRepository extends JpaRepository<Todo, Integer> {</pre>			

Auto table creation	Spring Boot + H2 + @Entity = automatic schema generation
data.sql	Pre-populates data at startup
spring.jpa.defer-datasource-initialization=true	Ensures tables exist before data.sql runs
camelCase → snake_case	Default JPA column naming convention

Next: Actually use JPA (via Spring Data JPA repositories) to **read/write** this data from the Controller/Service layer, replacing the old static list logic entirely.

54. Creating a Spring Data JPA Repository

Step 1: Create the TodoRepository interface

```
package
com.in28minutes.springboot.myfirstwebapp.todo;

import
org.springframework.data.jpa.repository.JpaRepository;

public interface TodoRepository extends
JpaRepository<Todo, Integer> {

}
```

Key points:

- Repository = an Interface, not a class** — Spring Data JPA generates the implementation automatically
- Naming convention: EntityName + Repository
- JpaRepository<Todo, Integer>** — generic parameters are <EntityType, IdType>. Since id is int, use the wrapper class Integer.

JpaRepository provides built-in operations out of the box:

```
List<Todo> findByUsername(String
username);
}
```

Key magic: Spring Data JPA parses the method name (findByUsername) and **automatically generates the query** based on the Todo entity's username field — zero SQL, zero implementation code required.

Step 5: Replace service call with repository call

```
@RequestMapping("list-todos")
public String listAllTodos(ModelMap model) {
    String username = getLoggedInUsername();
    model.addAttribute("todos",
    todoRepository.findByUsername(username));
    return "listTodos";
}
```

(Method naming was intentionally kept similar between TodoService and TodoRepository to make this swap easy.)

Error Encountered: "No default constructor for entity"

Cause: JPA/Hibernate requires entities to have a **no-argument constructor** (used internally for object creation/reflection), but Todo only had the all-fields constructor.

Fix — add a default constructor:

```
public Todo() {
}
```

(Keep the existing parameterized constructor too — both can coexist.)

Result

/list-todos now displays **4 Todos fetched directly from the H2 database** (populated via data.sql in Section 53) — confirming the full JPA pipeline works: Entity → Repository → Controller → View.

Key Takeaway

Concept

Repository = Interface

JpaRepository<Entity, IdType>

Method-name-based queries (findByUsername)

Entities need a no-arg constructor

Keeping old code as backup (@Controller commented out)

Next: Connect **Delete, Update, and Add Todo** operations to the database as well (currently only "list" is JPA-backed).

55. Connecting Delete, Update, and Add Todo to the Database

(Completing the JPA migration — all operations in TodoControllerJpa now use TodoRepository instead of the static list.)

Fix addNewTodo — use actual form values, not hardcoded ones

Before: Used hardcoded LocalDate.now().plusYears(1) and false regardless of user input.

After:

```
@RequestMapping(value = "add-todo", method =
RequestMethod.POST)
public String addNewTodo(ModelMap model,
@Valid Todo todo, BindingResult result) {
    if (result.hasErrors()) {
        return "todo";
    }

    String username = getLoggedInUsername();
```

Summary

No implementation code — Spring Data JPA generates it

Base interface providing full CRUD out of the box

Spring Data JPA auto-generates queries just from the method signature

Required by JPA/Hibernate internally

Safe way to experiment without breaking existing functionality or URL conflicts

```

    todo.setUsername(username);
    todoRepository.save(todo);
    return "redirect:list-todos";
}

```

- `todo.getTargetDate()` and `todo.isDone()` now come from the **actual submitted todo object** (thanks to 2-way binding — Section 44) instead of hardcoded values
- Only username needs to be set manually (not part of the form)
- **Why populate the whole todo object and call `.save()`?** `JpaRepository.save()` doesn't take individual field parameters — it expects a fully populated entity. So the pattern is: set remaining fields on the bean → call `.save(todo)`.

Fix `deleteTodo` — trivial swap

```

@RequestMapping("delete-todo")

public String deleteTodo(@RequestParam int id) {
    todoRepository.deleteById(id);
    return "redirect:list-todos";
}

```

Direct 1:1 replacement — `todoRepository.deleteById(id)` replaces the old service call.

Fix `showUpdateTodoPage` — retrieving by ID

```

@RequestMapping("update-todo")

public String
showUpdateTodoPage(@RequestParam int id,
ModelMap model) {
    Todo todo =
todoRepository.findById(id).get();
    model.addAttribute("todo", todo);
    return "todo";
}

```

Key detail: `findById()` returns an **Optional<Todo>**, not a `Todo` directly — must call `.get()` to extract the actual object.

Fix `updateTodo` — same pattern as `add`

```

@RequestMapping(value = "update-todo",
method = RequestMethod.POST)

```

```

public String updateTodo(@Valid Todo todo,
BindingResult result) {
    if (result.hasErrors()) {
        return "todo";
    }
    todoRepository.save(todo);
    return "redirect:list-todos";
}

```

Since the `todo` object already has its `id` (from the hidden field, Section 43) and all other fields populated from the form, calling `.save()` here performs an **update** (JPA's `save()` does insert-or-update based on whether the ID already exists).

Final Cleanup

Once all CRUD operations were migrated to `todoRepository`, the leftover `todoService` dependency was **completely removed**:

```

// Removed from TodoControllerJpa:

// - private TodoService todoService;

// - constructor parameter/assignment

```

`TodoControllerJpa` is now **fully JPA/database-backed**, with zero dependency on the old static-list-based `TodoService`.

Testing Confirmed

- **Add:** "Learn to Dance" with target date 30th June 2023 → saved correctly
- **Delete:** Removed AWS and Azure todos → confirmed gone
- **Update:** Changed "Get AWS Certified" → "Get AWS Certified 2" with new date → saved correctly

Key Takeaway

Operation	Old (Service/Static List)	New (Repository/JPA)
Add	<code>todoService.addTodo(...)</code> (individual params)	Populate <code>Todo</code> bean fully → <code>todoRepository.save(todo)</code>

Operation	Old (Service/Static List)	New (Repository/JPA)
Delete	todoService.deleteById(id)	todoRepository.deleteById(id)
Find by ID	Direct list search	todoRepository.findById(id).get() — returns Optional<Todo>
Update	Modify list in place	todoRepository.save(todo) — same method as Add (upsert behavior)

Big picture: In just 2 steps (Sections 54-55), the entire Todo app was migrated from an in-memory static list to a **real, persistent database** — with minimal code changes, thanks to Spring Data JPA's repository abstraction.

Next: Likely connecting to a real external database (**MySQL**) instead of the in-memory H2, as previewed back in Section 50.

56. Behind the Scenes — Viewing Generated SQL

(Conceptual/observational step — no new functionality, just visibility into what JPA does under the hood.)

Recap: What Spring Boot Auto Configuration Set Up

When spring-boot-starter-data-jpa + h2 were added (Section 50), Spring Boot automatically:

1. Initialized **JPA** and **Spring Data JPA** frameworks
2. Launched the **in-memory H2 database**
3. Set up a **connection pool** between the app and the database
4. Ran startup scripts like data.sql

Viewing the Generated SQL

```
spring.jpa.show-sql=true
```

With this enabled, the console logs show the **actual SQL** generated behind every repository

call — even though no SQL was written manually.

Observed Queries for Each Operation

Repository Call	Generated SQL (conceptually)
findByUsername(username)	SELECT * FROM todo WHERE username = ?
deleteById(id)	DELETE FROM todo WHERE id = ? (followed by a SELECT to refresh the list page)
findById(id)	SELECT * FROM todo WHERE id = ?
save(todo) (update case)	UPDATE todo SET description=?, done=?, target_date=?, username=? WHERE id=?

Also visible at startup: the SQL used to **create the sequence** (for @GeneratedValue IDs) and to **create the TODO table** itself.

Key Insight

Even though the app never writes SQL directly, **SQL is still very much happening in the background** — Spring Data JPA / Hibernate generates and executes it automatically based on the repository method calls.

This ties together everything from Sections 52-55: the "magic" of JPA isn't that SQL disappears — it's that **you no longer have to write it yourself**.

Cleanup

Since show-sql=true is mainly a debugging aid, it was **removed** at the end of the step (not meant to run permanently in normal use).

Limitation Reiterated

H2 is in-memory only — restarting the server **loses all data**. Fine for learning, **not suitable for production**.

Next: Switching from H2 to a real, persistent database — **MySQL**.

57. Switching from H2 to MySQL (using Docker)

Step 1: Launch MySQL via Docker

```
docker run -e
MYSQL_ROOT_PASSWORD=dummyspassword \
  -e MYSQL_USER=todos-user \
  -e MYSQL_PASSWORD=dummytodos \
  -e MYSQL_DATABASE=todos \
  --name mysql \
  -p 3306:3306 \
  mysql:<version>
```

(Exact command available in the course's GitHub repo, 02.Spring-Boot-Web-Application-V2 folder.)

Key parameters explained:

Parameter	Purpose
MYSQL_ROOT_PASSWORD	Root password for the DB server
MYSQL_USER / MYSQL_PASSWORD	Application's DB login credentials (todos-user / dummytodos)
MYSQL_DATABASE	Creates a database named todos at startup
-p 3306:3306	Exposes MySQL's default port (3306) to the host machine

Why Docker: Avoids installing MySQL directly on your machine — easy to spin up and **delete** without leftover system changes.

Verify: docker container ls → shows the running mysql container.

Step 2: Add MySQL dependency to pom.xml

```
<dependency>
```

```
<groupId>mysql</groupId>
<artifactId>mysql-connector-
java</artifactId>
</dependency>
```

Step 3: Update application.properties

```
# H2 settings — commented out
# spring.datasource.url=jdbc:h2:mem:testdb

# MySQL settings
spring.datasource.url=jdbc:mysql://localhost:3306/todos
spring.datasource.username=todos-user
spring.datasource.password=dummytodos

spring.jpa.properties.hibernate.dialect=org.
hibernate.dialect.MySql8Dialect

spring.jpa.hibernate.ddl-auto=update
```

Key properties explained:

Property	Purpose
spring.datasource.url	JDBC URL — note the format is jdbc:mysql://..., not http://
spring.datasource.username / password	Matches the Docker-created MySQL user
spring.jpa.properties.hibernate.dialect	Tells Hibernate to generate MySQL-specific SQL (differs slightly from H2's SQL dialect)
spring.jpa.hibernate.ddl-auto=update	Critical for real databases — H2 auto-creates tables by default,

Property	Purpose
	but MySQL does NOT; this property tells Hibernate to create/update the schema at startup

Important Behavior Change

data.sql does NOT run when connecting to a real database like MySQL (it only auto-executes for in-memory databases like H2). So after switching, the Todos list starts **empty** — must Add Todos manually to test.

Verifying the Connection via MySQL Shell (mysqlsh)

```
\connect todos-user@localhost:3306
```

```
[enter password: dummytodos]
```

```
\use todos
```

```
\sql
```

```
select * from todo;
```

Confirms that Todos added through the web app (<http://localhost:8080>) actually appear in the **real MySQL database**.

Key Advantage Demonstrated

Unlike H2, **MySQL data persists across application restarts**. Stopping and restarting the Spring Boot app does **not** wipe the Todos — querying `select * from todo` afterward still returns the same data.

Key Takeaway

Aspect	H2 (in-memory)	MySQL (via Docker)
Data persistence	Lost on restart	Persists across restarts
Table creation	Automatic	Requires <code>spring.jpa.hibernate.ddl-auto=update</code>

Aspect	H2 (in-memory)	MySQL (via Docker)
data.sql execution	Runs automatically	Does not run
Use case	Development/testing/learning	Production-like setup

Big Picture — What Changed to Migrate H2 → MySQL

Only **2 files** needed changes:

1. `pom.xml` — swap H2 dependency for `mysql-connector-java`
2. `application.properties` — update `datasource URL/credentials`, `dialect`, and `ddl-auto`

No Java code changes were needed — this demonstrates the power of the **JPA/Spring Data JPA abstraction** (Sections 52-55): the application logic (`TodoRepository`, `TodoControllerJpa`) is completely **database-agnostic**.

This appears to be a natural milestone/wrap-up point for the Todo Management Application section of the course (H2 → MySQL was the last item flagged back in Section 50). Let me know when you have the next transcript.